

ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

—o0o—



LAN TRUYỀN NGƯỢC THEO THỜI GIAN TRONG
MẠNG NƠ-RON HỒI TIẾP

(Recurrent Neural Network's Backpropagation Through Time (RNN BPTT))

Khoa: Khoa Học và Kỹ Thuật Máy Tính

Giảng viên hướng dẫn: TS. Nguyễn An Khương

TS. Trần Tuấn Anh

Học viên thực hiện: Đỗ Thanh Thái - 2480860

Nguyễn Minh Quân - 2212804

Nguyễn Quỳnh Như - 2212472

Nguyễn Tấn Phú - 2480859

Phạm Lê Ngọc Sơn - 2470741

Phạm Duy Tùng - 2470753

Báo cáo bài tập lớn

Môn học: Cơ sở Toán cho Khoa Học Máy Tính

TP. HỒ CHÍ MINH, tháng 05 năm 2025

Đường dẫn đến Github của nhóm:

- Đường dẫn tới: Github

LỜI CẢM ƠN

Trong suốt quá trình học môn Cơ Sở Toán Cho Khoa Học Máy Tính, nhóm chúng em đã nhận được rất nhiều kiến thức bổ ích từ Thầy TS. Nguyễn An Khương và Thầy TS. Trần Tuấn Anh; nhóm chúng em thực sự trân trọng điều đó. Những bài giảng của Quý Thầy đã giúp nhóm chúng em xây dựng một nền tảng vững chắc trong việc hiểu, phân biệt, ứng dụng cụ thể của từng Cơ sở Toán học trong Khoa học máy tính.

Nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc tới Thầy TS. Nguyễn An Khương và Thầy TS. Trần Tuấn Anh, người đã luôn tận tâm, tận tình chỉ bảo và truyền đạt những kiến thức quý báu trong suốt quá trình học. Những lời hướng dẫn của Quý Thầy không chỉ giúp nhóm chúng em trang bị thêm kiến thức cho công việc học tập và nghiên cứu, mà còn có ảnh hưởng lớn trong cuộc sống và sự nghiệp của nhóm chúng em hiện tại và trong tương lai.

Cuối cùng, nhóm chúng em xin kính chúc Quý Thầy luôn mạnh khỏe, an vui và tràn đầy năng lượng để tiếp tục cống hiến cho sự nghiệp giáo dục, truyền tải tri thức cho các thế hệ sau.

TP. HCM, ngày 30 tháng 04 năm 2025

Nhóm học viên thực hiện

*Đại diện nhóm báo cáo.
Học viên. Đỗ Thanh Thái*

Thành viên nhóm và phân công công việc

MSHV	Họ và tên	Tỉ lệ hoàn thành (%)	Công việc thực hiện
2480860	Đỗ Thanh Thái	100%	Mathematical Foundation of BPTT (.ipynb). • Backpropagation Through Time in Detail. • Derivation of gradients. • Analysis of Gradients in RNNs. • Memory considerations: storing intermediate values across time steps. • Vanishing and exploding gradient problems. • Comparison with Other Training Techniques. • Limitations, Research Trends, and Future Directions. Viết báo cáo (.pdf).
2212804	Nguyễn Minh Quân	100%	Implementation Details and Challenges: Truncating Time Steps (.ipynb).
2212472	Nguyễn Quỳnh Như	100%	Introduction to Recurrent Neural Networks (RNNs) and the Need for BPTT (.ipynb).
2480859	Nguyễn Tấn Phú	100%	Implementation Details and Challenges: Randomized Truncation (.ipynb).
2470741	Phạm Lê Ngọc Sơn	100%	Implementation Details and Challenges: Full BPTT (.ipynb).
2470753	Phạm Duy Tùng	100%	Applications of BPTT (.ipynb).

Mục lục

1	Mạng Nơ-ron Hồi tiếp và Sự Cần thiết của BPTT	9
1.1	Neural Network	9
1.2	Hạn chế của Neural Network truyền thống với dữ liệu tuần tự theo thời gian	10
1.3	Recurrent Neural Network	11
1.3.1	Dữ liệu tuần tự theo thời gian	11
1.3.2	Tổng quan về Recurrent Neural Network	12
1.4	Vấn đề Backpropagation trong Recurrent Neural Network	14
1.4.1	Thuật toán Backpropagation hoạt động không tốt trên RNN	14
1.4.2	Sự cần thiết của Backpropagation Through Time	15
2	Nền tảng Toán học của Lan truyền ngược qua thời gian	16
2.1	So sánh giữa Feedforward Neural Networks, Backpropagation, và Backpropagation Through Time trong RNN (BPTT-RNN)	16
2.1.1	Feedforward Neural Network (FFN)	16
2.1.2	Backpropagation (BP) thông thường	17
2.1.3	Backpropagation Through Time (BPTT) cho RNN	18
2.1.4	So sánh một số công thức cơ bản của Feedforward và Backpropagation trong quá trình huấn luyện mạng nơ-ron nhân tạo	21
2.2	Nền tảng Toán học	22
2.2.1	Lan truyền ngược qua thời gian (Backpropagation Through Time - BPTT)	22
2.2.2	Tối ưu hóa RNN bằng Gradient Descent	23
2.3	Các vấn đề tiềm ẩn như vanishing/exploding gradients	24

2.3.1	Bài toán minh họa: Đếm số lượng ký tự “1” trong chuỗi nhị phân bằng RNN	24
2.3.2	Mô tả quá trình huấn luyện RNN	25
2.3.3	Mô tả hiện tượng Bùng nổ Gradient trong RNN	26
2.3.4	Phân tích nguyên nhân xảy ra Bùng nổ gradient trong mạng RNN	28
3	Giải pháp và Các phương pháp huấn luyện khác	30
3.1	Giải pháp xử lý vanishing/exploding gradients bằng Truncated BPTT	30
3.1.1	Cơ sở toán học	31
3.1.2	BPTT đầy đủ (Full BPTT)	32
3.1.3	BPTT cắt ngắn (Truncated BPTT) [1]	33
3.1.4	Ví dụ	33
3.1.5	Truncated BPTT: Randomized Truncation vs. Standard Truncation	34
3.2	Các phương pháp huấn luyện khác	38
3.2.1	Gradient Clipping	38
3.2.2	BPTT trong LSTM và GRU	39
3.2.3	Phương pháp không dùng gradient (Gradient-free methods)	42
3.2.4	Real-Time Recurrent Learning (RTRL)	44
4	Các ứng dụng của BPTT	46
4.1	Các ứng dụng trong các lĩnh vực khác nhau	46
4.1.1	Xử lý ngôn ngữ tự nhiên (NLP)	46
4.1.2	Dự báo chuỗi thời gian	46
4.1.3	Robotics	47
4.1.4	Học tăng cường (Reinforcement Learning)	47
5	Hạn chế, Xu hướng nghiên cứu và Định hướng tương lai	48
5.1	Những hạn chế hiện tại của RNN và BPTT	48
5.2	Các hướng tiếp cận thay thế: Cơ chế Attention và Transformer	49
5.3	Xu hướng nghiên cứu gần đây	49
5.3.1	Định hướng tương lai	49

6	Phụ lục	51
6.1	Perceptron đơn (Single Perceptron):	51
6.2	Mạng nơ-ron nhiều lớp (Multilayer Neural Network) [2]	51
6.3	Incremental Gradient Descent và Các biến thể	52
6.3.1	Batch Gradient Descent [2]	52
6.3.2	Mini-Batch Gradient Descent [2]	53
6.3.3	Momentum [3]	54
6.3.4	Nesterov Accelerated Gradient (NAG) [2]	54
6.3.5	Adagrad [4]	55
6.3.6	RMSProp [5]	55
6.3.7	Adam (Adaptive Moment Estimation) [4]	55
6.3.8	Các biến thể khác: AdaDelta / Nadam / AMSGrad [6]	55
6.4	Thuật toán Backpropagation	55
6.5	Quy tắc đạo hàm của hàm Lan truyền ngược (Backpropagation)	59
6.5.1	Hàm lỗi	59
6.5.2	Cập nhật trọng số với Gradient Descent	59
6.5.3	Đạo hàm theo trọng số	59
6.5.4	Quy tắc huấn luyện đối với đơn vị đầu ra	59
6.5.5	Quy tắc huấn luyện đối với trọng số của đơn vị ẩn	60
6.5.6	Tính toán $E[z_t]$	61
6.5.7	Các đại lượng xác định (không ngẫu nhiên)	62
6.5.8	Mở rộng cơ sở toán: Lan truyền ngược theo thời gian trong RNN	64
6.5.9	Các tham số trong RNN	67
6.5.10	Stacked RNN (RNN nhiều tầng) [7]	67
6.5.11	Các kiểu kiến trúc RNN	68
	Tài liệu tham khảo	73

Danh sách bảng

2.1	So sánh giữa FFN + BP và RNN + BPTT	21
2.2	So sánh giữa Feedforward và Backpropagation trong mạng nơ-ron nhiều lớp	22
3.1	So sánh <i>Full BPTT</i> và <i>Truncated BPTT</i>	30
3.2	Tổng quát hóa các tình huống sử dụng Full Truncated BPTT . . .	34
3.3	So sánh <i>Randomized Truncation</i> và <i>Standard Truncation</i>	34
3.4	Tổng quát hóa các tình huống sử dụng Randomized Standard Truncation	38
3.5	So sánh luồng gradient trong BPTT của các mô hình	41
3.6	Đánh giá các phương pháp tối ưu phổ biến không dùng gradient .	43
3.7	Tổng quan: BPTT so với RTRL	44
3.8	Các điểm khác biệt chính giữa BPTT và RTRL	45
3.9	Lựa chọn sử dụng BPTT và RTRL	45
6.1	Giá trị đầu vào và trọng số ban đầu	57
6.2	Tính toán đầu vào và đầu ra	58
6.3	Tính toán lỗi tại mỗi nút	58
6.4	Tính toán cập nhật trọng số	58

Danh sách hình vẽ

1.1	Mô hình Neural Network đơn giản. [8]	10
1.2	Ví dụ về RNN đơn giản. [9]	13
2.1	Mạng Feedforward Neural Networks	17
2.2	Thuật toán lan truyền ngược - Backpropagation	18
2.3	Forward Pass (của BPTT) trong RNN	19
2.4	Backward Pass (của BPTT) trong RNN	20
2.5	Kết quả huấn luyện với đầu vào gồm 3 số “1”.	26
2.6	Xảy ra hiện tượng exploding trong quá trình huấn luyện.	27
6.1	Ranh giới quyết định của Perceptron tuyến tính.	52
6.2	Ranh giới quyết định của Perceptron nhiều lớp.	53
6.3	Ranh giới quyết định của Perceptron nhiều lớp có sử dụng thuật toán lan truyền ngược.	54
6.4	So sánh gradient descent variants đối với hàm bậc 2.	56
6.5	Một mạng nơ-ron đơn giản.	57

Giới thiệu

Động lực nghiên cứu

Trong vài thập kỷ qua, mạng nơ-ron nhân tạo đã đạt được những bước tiến vượt bậc trong nhiều lĩnh vực như xử lý ngôn ngữ tự nhiên (NLP), thị giác máy tính, và điều khiển robot. Tuy nhiên, hầu hết các mạng nơ-ron truyền thống chỉ xử lý dữ liệu đầu vào tĩnh, không phù hợp với các bài toán mang tính chất tuần tự như ngôn ngữ, chuỗi thời gian hay tín hiệu cảm biến.

Mạng nơ-ron hồi tiếp (Recurrent Neural Network - RNN) được phát triển để giải quyết các bài toán trên bằng cách cho phép mô hình có khả năng "ghi nhớ" thông tin từ các bước thời gian trước. Tuy nhiên, việc huấn luyện RNN đặt ra những thách thức lớn do sự phụ thuộc giữa các thời điểm trong chuỗi. Điều này đòi hỏi một phương pháp lan truyền lỗi hiệu quả qua thời gian — được gọi là **Backpropagation Through Time (BPTT)**.

Mục tiêu của bài phân tích

Bài phân tích này vừa là bài tập lớn và cũng là tài liệu được biên soạn nhằm mục tiêu cung cấp một cái nhìn tổng quan về:

- Nguyên lý hoạt động của BPTT và các mở rộng của thuật toán lan truyền ngược (Backpropagation) cho dữ liệu tuần tự.
- Các vấn đề kỹ thuật như *vanishing gradient* và *exploding gradient* thường gặp trong huấn luyện RNN.
- Những thông tin về định hướng phát triển mới, bao gồm kết hợp BPTT với các mô hình hiện đại như Attention, Transformer, hoặc mạng thần kinh có bộ nhớ.
- Các ứng dụng thực tiễn trong nhiều lĩnh vực như xử lý ngôn ngữ tự nhiên, dự báo chuỗi thời gian, robot và học tăng cường.

Cấu trúc tài liệu

Bài tập lớn được tổ chức thành các chương như sau:

- **Chương 1:** Mạng Nơ-ron Hồi tiếp và Sự Cần thiết của Backpropagation Through Time (BPTT).
- **Chương 2:** Nền tảng Toán học của Lan truyền ngược qua thời gian.
- **Chương 3:** Giải pháp và Các phương pháp huấn luyện khác.
- **Chương 4:** Các ứng dụng của BPTT.
- **Chương 5:** Hạn chế, Xu hướng nghiên cứu và Định hướng tương lai.
- **Chương 6:** Phụ lục.

Tầm quan trọng của BPTT

Trong bối cảnh ngày càng có nhiều ứng dụng yêu cầu xử lý thông tin theo thời gian như trò chuyện thông minh, dự báo tài chính, và điều khiển thiết bị thông minh, việc hiểu và cải tiến phương pháp BPTT trở nên cấp thiết. Việc nắm vững kiến thức về BPTT không chỉ giúp cải thiện hiệu suất mô hình mà còn mở đường cho những đột phá trong trí tuệ nhân tạo.

Chương 1

Mạng Nơ-ron Hồi tiếp và Sự Cần thiết của BPTT

1.1 Neural Network

Giới thiệu

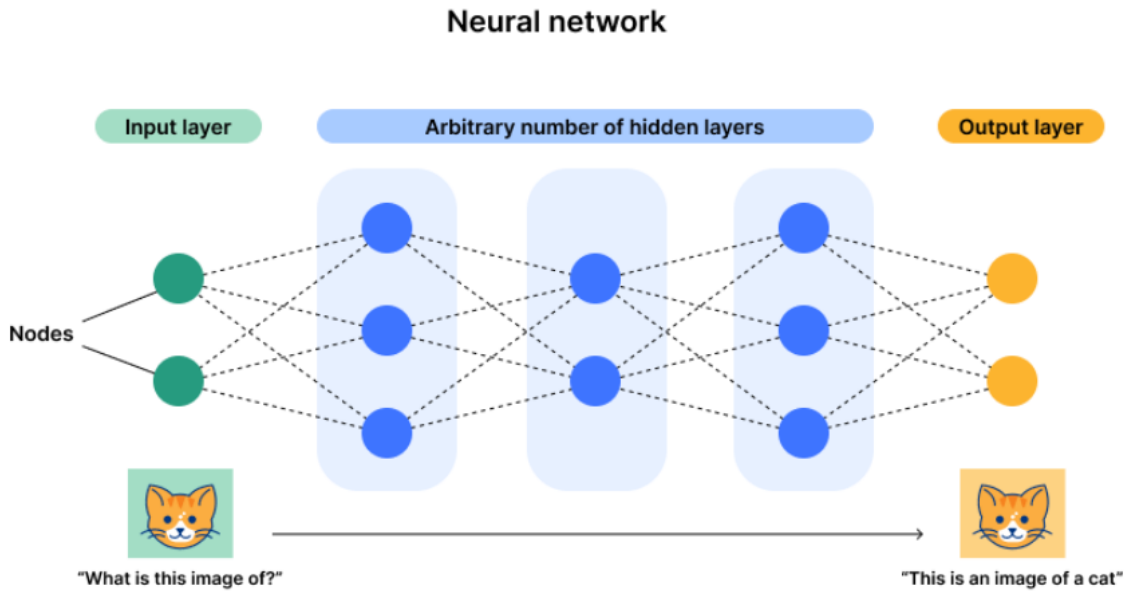
Neural Network [10] là một mô hình học máy mô phỏng hoạt động tương tự hệ thần kinh của con người. Neural Network bao gồm các lớp neuron liên kết với nhau bằng các trọng số, được thể hiện trên hình 1.1, với cơ chế hoạt động:

- Nhận đầu vào (input).
- Tính toán tổng có trọng số + bias.
- Áp dụng hàm kích hoạt (activation function).
- Xuất ra giá trị dự đoán (output).

Trong quá trình huấn luyện, thuật toán Backpropagation được sử dụng để cập nhật trọng số của mạng, sao cho đầu ra tiệm cận giá trị mong muốn.

Ví dụ mô hình đơn giản:

Input $\rightarrow$ Hidden Layer $\rightarrow$ Output Layer



Hình 1.1: Mô hình Neural Network đơn giản. [8]

1.2 Hạn chế của Neural Network truyền thống với dữ liệu tuần tự theo thời gian

Neural Network truyền thống chỉ có thể xử lý thông tin theo một chiều từ Input $\rightarrow$ Output. Công thức chung:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (1.1)$$

Trong đó:

- $\mathbf{h}^{(l)}$ là vector output tầng l .
- $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$ là ma trận trọng số và bias.
- σ là hàm kích hoạt (ReLU, tanh, sigmoid...).

Chính vì thông tin chỉ đi được một chiều mà không thể quay lại (do không có vòng lặp), các input tại các node không thể nhớ thông tin nào từ trước. Bên cạnh đó, nếu xử lý nhiều input, thì các input này cũng được xử lý độc lập và không biết giá trị trước sau. Cụ thể:

Giả sử với chuỗi input:

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_T \quad (1.2)$$

Với Neural Network truyền thống:

$$\mathbf{y}_t = f(\mathbf{x}_t) \quad (1.3)$$

Nhưng thực tế cần xử lý tuần tự:

$$\mathbf{y}_t = f(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t) \quad (1.4)$$

Trong thực tế, rất nhiều bài toán yêu cầu mô hình phải “nhớ” được các dữ liệu trong quá khứ, liên kết chúng lại và dự đoán giá trị output. Chẳng hạn như các bài toán sau đều không thể chỉ xử lý từng input đơn lẻ mà cần xem xét chuỗi các input liên tiếp:

- **Dịch máy (Machine Translation):** Nghĩa của một từ phụ thuộc vào các từ trước đó.
- **Dự đoán giá cổ phiếu:** Giá hiện tại bị ảnh hưởng bởi xu hướng giá trước đó.
- **Nhận diện giọng nói:** Âm thanh ở thời điểm hiện tại phụ thuộc ngữ cảnh âm thanh trước đó.
- **Chatbot hội thoại:** Câu trả lời hiện tại phải dựa trên lịch sử đoạn hội thoại.

Tóm lại: Neural Network truyền thống được thiết kế để xử lý các dữ liệu đầu vào độc lập, không có mối liên kết theo thời gian giữa các bước. Mô hình sẽ hoạt động hiệu quả khi dữ liệu đầu vào đã hoàn chỉnh và không mang tính chuỗi như trong các bài toán phân loại, hồi quy. Tuy nhiên, với dữ liệu tuần tự, phụ thuộc vào các thông tin trước đó, Neural Network hoàn toàn không có cơ chế ghi nhớ trạng thái hay ngữ cảnh. Chính vì thế, *Recurrent Neural Network* ra đời, với cơ chế hidden state được cập nhật liên tục qua mỗi bước thời gian, cho phép mô hình có thể lưu trữ và khai thác thông tin từ quá khứ, phục vụ cho việc xử lý chuỗi dữ liệu một cách tự nhiên và hiệu quả hơn.

1.3 Recurrent Neural Network

1.3.1 Dữ liệu tuần tự theo thời gian

Trong nhiều bài toán thực tế, dữ liệu không chỉ đơn thuần độc lập mà còn mang bản chất tuần tự. Tức là, giá trị tại thời điểm hiện tại không tồn tại riêng

lẻ, mà chịu ảnh hưởng bởi những dữ liệu xảy ra trước đó. Hiểu một cách đơn giản, để đưa ra quyết định ở thời điểm hiện tại, mô hình cần phải xem xét không chỉ dữ liệu hiện tại mà còn cả lịch sử các sự kiện trước đó.

Chẳng hạn, trong xử lý ngôn ngữ tự nhiên (NLP), khi đọc một câu văn, việc hiểu nghĩa của một từ phụ thuộc rất nhiều vào các từ đi trước. Ví dụ:

- Câu: "Hôm nay trời rất ____."
 - Nếu các từ trước đó là "Hôm nay trời rất", thì từ tiếp theo hợp lý có thể là "đẹp", "nắng", "mát", ...
 - Nếu chỉ nhìn từ "rất", sẽ không thể đoán chính xác được ý nghĩa mong muốn.

Để có thể xử lý loại dữ liệu trên, một mô hình cần có các đặc điểm sau:

- **Tiếp nhận một đầu vào tại mỗi thời điểm:** Ví dụ: một từ trong câu, một ảnh từ camera, một giá trị cảm biến đo được.
- **Cập nhật trạng thái ẩn (hidden state):**
 - Trạng thái ẩn đóng vai trò là bộ nhớ, lưu giữ những thông tin quan trọng từ các thời điểm trước.
 - Hidden state được cập nhật dựa trên đầu vào hiện tại kết hợp với trạng thái trước đó.
- **Sinh ra đầu ra tương ứng:**
 - Output tại thời điểm hiện tại có thể phụ thuộc không chỉ vào input hiện tại mà còn phụ thuộc vào toàn bộ lịch sử các input trước.

1.3.2 Tổng quan về Recurrent Neural Network

Recurrent Neural Network (RNN) [11–13] được thiết kế để xử lý dữ liệu tuần tự bằng cách giới thiệu trạng thái ẩn (hidden state) để lưu trữ thông tin về các input trước đó, được thể hiện qua hình 1.2.

Tại mỗi bước thời gian t , RNN thực hiện các bước sau:

- Nhận đầu vào $\mathbf{X}_t$,
- Kết hợp đầu vào hiện tại $\mathbf{X}_t$ và trạng thái ẩn từ bước trước $\mathbf{H}_{t-1}$,

- Cập nhật trạng thái ẩn $\mathbf{H}_t$,
- Sinh ra đầu ra $\mathbf{O}_t$ dựa trên trạng thái ẩn $\mathbf{H}_t$.

Cập nhật trạng thái ẩn:

$$\mathbf{H}_t = \varphi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h) \quad (1.5)$$

Trong đó:

- $\mathbf{X}_t \in \mathbb{R}^{n \times d}$: Input tại thời điểm t (vector hoặc batch).
- $\mathbf{H}_t \in \mathbb{R}^{n \times h}$: Hidden state tại thời điểm t .
- $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$: Trọng số từ input đến hidden.
- $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$: Trọng số từ hidden trước tới hidden hiện tại.
- $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$: Bias cho hidden state.
- φ : Hàm kích hoạt (thường là tanh hoặc ReLU).

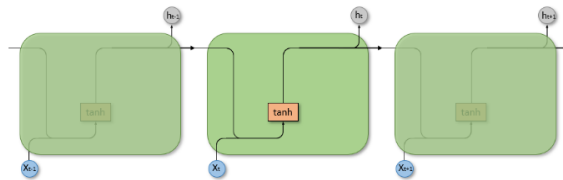
Output sinh ra:

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q \quad (1.6)$$

Trong đó:

- $\mathbf{O}_t \in \mathbb{R}^{n \times q}$: Output tại thời điểm t .
- $\mathbf{W}_{hq} \in \mathbb{R}^{h \times q}$: Trọng số từ hidden đến output.
- $\mathbf{b}_q \in \mathbb{R}^{1 \times q}$: Bias cho output.

Lưu ý: Với mô hình ở các bước thời gian khác nhau, trọng số và bias là giống nhau. Chính vì thế, khi số lượng bước thời gian tăng lên, mô hình không cần thêm tham số mới, đảm bảo kích thước mô hình cố định bất kể độ dài của chuỗi đầu vào.



Hình 1.2: Ví dụ về RNN đơn giản. [9]

1.4 Vấn đề Backpropagation trong Recurrent Neural Network

Trong mạng Neural Network truyền thống, quá trình học sử dụng thuật toán backpropagation [11] đơn giản, chỉ lan truyền lỗi ngược qua từng lớp (từ đầu ra về đầu vào). Ngoài ra, các trọng số ở mỗi lớp là riêng biệt và dữ liệu được xử lý độc lập, không liên quan đến yếu tố thời gian.

Tuy nhiên, trong mạng RNN, do mô hình xử lý dữ liệu theo chuỗi thời gian và các trọng số được chia sẻ qua các bước thời gian, quá trình tính toán gradient phải lan truyền không chỉ qua các lớp, mà còn xuyên suốt qua các bước thời gian. Phương pháp lan truyền lỗi ngược trong RNN được gọi là **Backpropagation Through Time (BPTT)** [14].

1.4.1 Thuật toán Backpropagation hoạt động không tốt trên RNN

Khi áp dụng trực tiếp thuật toán backpropagation cho RNN, hai vấn đề lớn xuất hiện do đặc điểm tuần tự của dữ liệu:

(1) Vanishing Gradient - Triệt tiêu Gradient

- Khi gradient được truyền ngược qua rất nhiều bước thời gian, mỗi bước đều nhân với ma trận trọng số và đạo hàm của hàm kích hoạt.
- Nếu giá trị của đạo hàm hoặc trọng số nhỏ hơn 1, việc nhân liên tục sẽ làm cho gradient giảm dần về 0.
- Hậu quả:
 - Các trọng số liên quan đến các bước thời gian xa hầu như không được cập nhật.
 - Mô hình không học được các mối quan hệ dài hạn trong dữ liệu (*long-term dependencies*).
- Ví dụ: khi dịch một câu quá dài, mô hình không thể nhớ được những từ đầu tiên, dẫn đến bản dịch bị mất ngữ cảnh.

(2) Exploding Gradient - Bùng nổ Gradient

- Ngược lại, nếu giá trị của đạo hàm hoặc trọng số lớn hơn 1, gradient sẽ tăng nhanh chóng khi nhân liên tiếp qua nhiều bước.

- Hậu quả:
 - Gradient trở nên rất lớn, khiến quá trình cập nhật trọng số mất kiểm soát.
 - Việc huấn luyện trở nên không ổn định, có thể dẫn đến **overflow** hoặc **NaN**.
- Ví dụ: mô hình đưa ra các dự đoán không hợp lý, nhảy lung tung hoặc không hội tụ trong quá trình huấn luyện.

1.4.2 Sự cần thiết của Backpropagation Through Time

Trong quá trình huấn luyện mô hình với dữ liệu tuần tự, tại mỗi thời điểm t , mô hình không chỉ xử lý đầu vào mới $\mathbf{X}_t$, mà còn phải cập nhật trạng thái ẩn dựa trên thông tin hiện tại và trạng thái tích lũy từ các bước thời gian trước.

Việc duy trì và cập nhật trạng thái ẩn theo thời gian là yếu tố then chốt giúp mô hình học được các quan hệ dài hạn trong chuỗi dữ liệu.

Do đó, để cập nhật tham số đúng cách, quá trình tính gradient không thể chỉ diễn ra theo lớp như mạng truyền thống, mà phải lan truyền qua toàn bộ chuỗi thời gian. Cơ chế này đảm bảo rằng:

- Thông tin từ quá khứ không bị "quên".
- Mỗi quan hệ theo thời gian được học một cách hiệu quả.

Tuy nhiên, lan truyền gradient qua nhiều bước liên tiếp gây ra hiện tượng **triệt tiêu gradient** và **bùng nổ gradient** đã đề cập ở trên. Điều này dẫn đến:

- Mô hình quên thông tin đầu chuỗi khi chuỗi quá dài.
- Mô hình dự đoán sai hoặc huấn luyện không ổn định.

Chính vì thế, huấn luyện RNN đòi hỏi kỹ thuật đặc biệt gọi là **BPTT**, cho phép mở rộng mạng theo thời gian và tính toán gradient chính xác trên toàn bộ chuỗi.

Chương 2 sẽ phân tích về nền tảng toán học của lan truyền ngược qua thời gian.

Chương 2

Nền tảng Toán học của Lan truyền ngược qua thời gian

2.1 So sánh giữa Feedforward Neural Networks, Backpropagation, và Backpropagation Through Time trong RNN (BPTT-RNN)

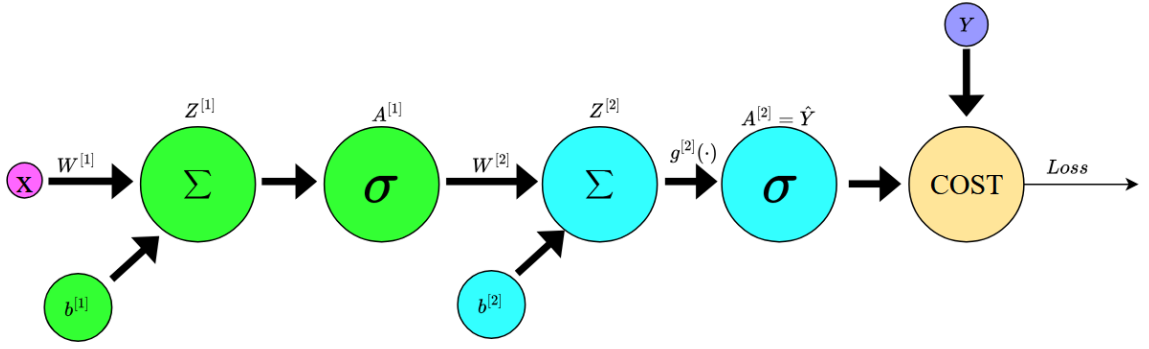
2.1.1 Feedforward Neural Network (FFN)

Truyền xuôi (Forward Pass):

- Dữ liệu đi từ input $\rightarrow$ hidden layer(s) $\rightarrow$ output.
- Công thức chính:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}, \quad A^{[l]} = g^{[l]}(Z^{[l]})$$

- Output ($A^{[L]}$) so sánh với label Y để tính loss L .



Hình 2.1: Mạng Feedforward Neural Networks

Hình 2.1 mô tả mạng Feedforward Neural Networks, với các công thức tính toán từ 2.1 đến 2.5.

$$Z^{[1]} = W^{[1]}X + b^{[1]} \quad (2.1)$$

$$A^{[1]} = g^{[1]}(Z^{[1]}) \quad (2.2)$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} \quad (2.3)$$

$$A^{[2]} = \hat{Y} = g^{[2]}(Z^{[2]}) \quad (2.4)$$

$$L = \text{Cost}(A^{[2]}, Y) \quad (2.5)$$

2.1.2 Backpropagation (BP) thông thường

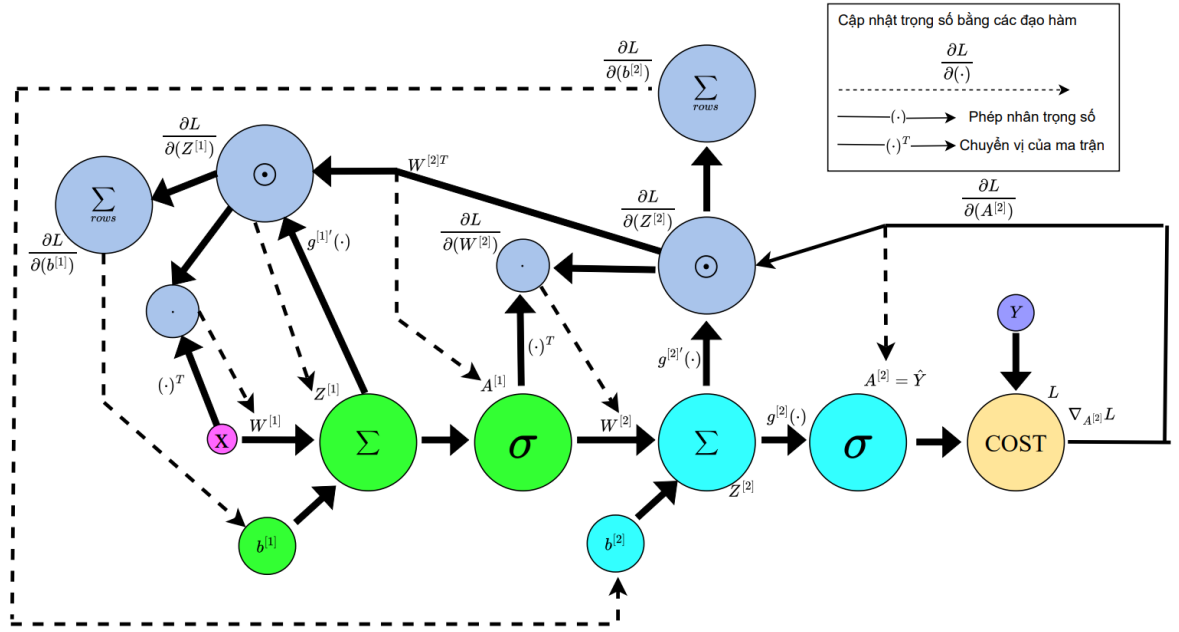
Truyền ngược (Backward Pass):

- Tính gradient của loss L so với từng tham số (W, b) bằng quy tắc đạo hàm hàm hợp.

- Công thức chính:

$$\frac{\partial L}{\partial Z^{[l]}} = \frac{\partial L}{\partial A^{[l]}} \odot g^{[l]'}(Z^{[l]}), \quad \frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial Z^{[l]}} A^{[l-1]T}$$

- **Mục đích:** Cập nhật W, b để giảm loss.



Hình 2.2: Thuật toán lan truyền ngược - Backpropagation

Hình 2.2 mô tả thuật toán lan truyền ngược, với các công thức tính toán từ 2.6 đến 2.13.

$$\frac{\partial L}{\partial A^{[2]}} = \nabla_{A^{[2]}} \text{Cost} \quad (2.6)$$

$$\frac{\partial L}{\partial Z^{[2]}} = \frac{\partial L}{\partial A^{[2]}} \odot g^{[2]'}(Z^{[2]}) \quad (2.7)$$

$$\frac{\partial L}{\partial W^{[2]}} = \frac{\partial L}{\partial Z^{[2]}} A^{[1]T} \quad (2.8)$$

$$\frac{\partial L}{\partial b^{[2]}} = \sum_{\text{rows}} \frac{\partial L}{\partial Z^{[2]}} \quad (2.9)$$

$$\frac{\partial L}{\partial A^{[1]}} = W^{[2]T} \frac{\partial L}{\partial Z^{[2]}} \quad (2.10)$$

$$\frac{\partial L}{\partial Z^{[1]}} = \frac{\partial L}{\partial A^{[1]}} \odot g^{[1]'}(Z^{[1]}) \quad (2.11)$$

$$\frac{\partial L}{\partial W^{[1]}} = \frac{\partial L}{\partial Z^{[1]}} X^T \quad (2.12)$$

$$\frac{\partial L}{\partial b^{[1]}} = \sum_{\text{rows}} \frac{\partial L}{\partial Z^{[1]}} \quad (2.13)$$

2.1.3 Backpropagation Through Time (BPTT) cho RNN

Khác biệt so với BP thông thường:

- Mạng RNN có trạng thái ẩn ($A_t^{[1]}$) phụ thuộc vào thời gian t .

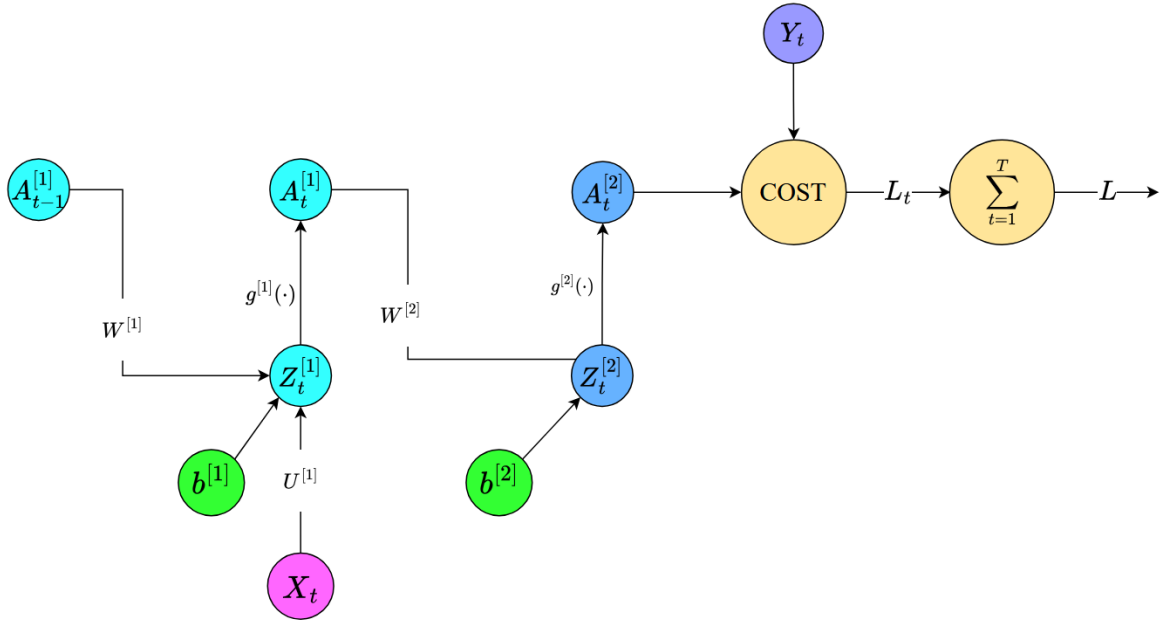
- Unfold RNN qua các bước thời gian ($t = 1$ đến T).

Công thức chính:

- Gradient tại bước t phụ thuộc cả **output hiện tại** và **trạng thái ẩn tương lai** ($t + 1$):

$$\frac{\partial L}{\partial A_t^{[1]}} = W^{[2]T} \frac{\partial L}{\partial Z_t^{[2]}} + W^{[1]T} \frac{\partial L}{\partial Z_{t+1}^{[1]}}$$

- Cộng dồn gradient của $W^{[1]}, U^{[1]}, b^{[1]}$ qua **tất cả các bước thời gian**.



Hình 2.3: Forward Pass (của BPTT) trong RNN

Hình 2.3 mô tả Forward Pass (của BPTT) trong RNN, với các công thức tính toán từ 2.14 đến 2.19.

$$Z_t^{[1]} = W^{[1]} A_{t-1}^{[1]} + U^{[1]} X_t + b^{[1]} \quad (2.14)$$

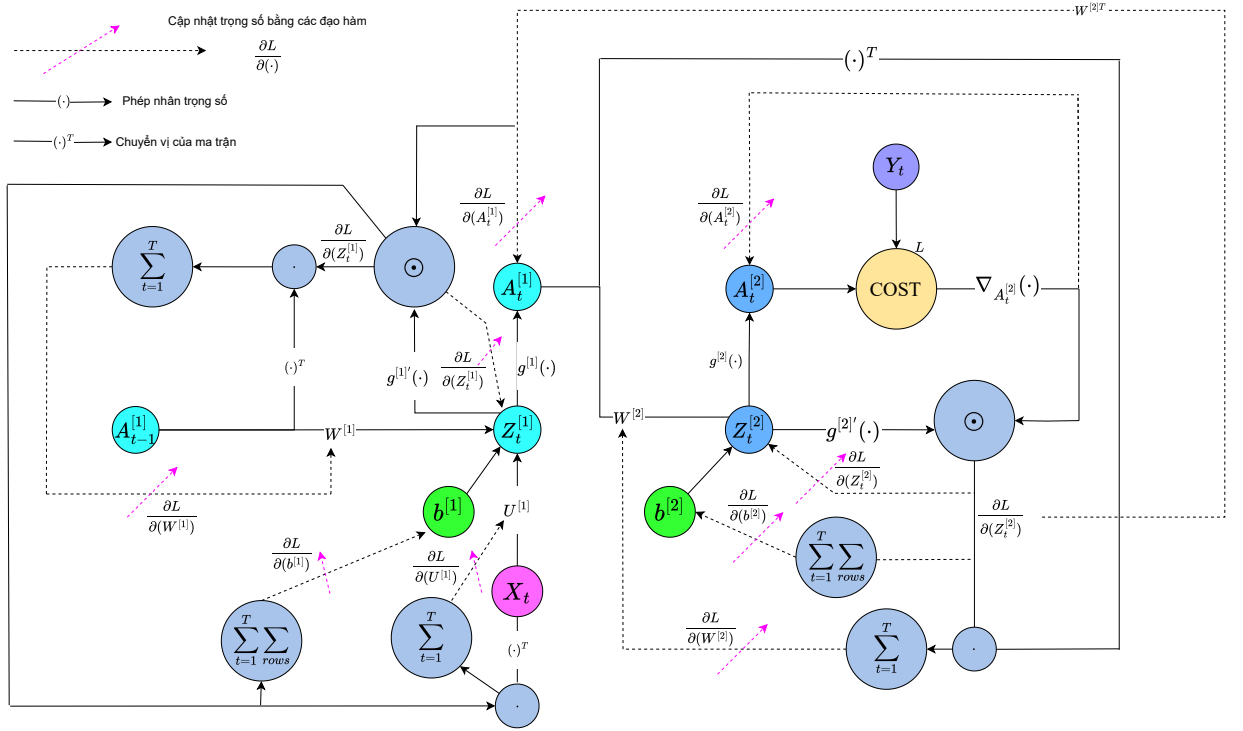
$$A_t^{[1]} = g^{[1]}(Z_t^{[1]}) \quad (2.15)$$

$$Z_t^{[2]} = W^{[2]} A_t^{[1]} + b^{[2]} \quad (2.16)$$

$$A_t^{[2]} = \hat{Y}_t = g^{[2]}(Z_t^{[2]}) \quad (2.17)$$

$$L_t = \text{Cost}(A_t^{[2]}, Y_t) \quad (2.18)$$

$$L = \sum_{t=1}^T L_t \quad (2.19)$$



Hình 2.4: Backward Pass (của BPTT) trong RNN

Hình 2.4 mô tả Backward Pass (của BPTT) trong RNN, với các công thức tính toán từ 2.20 đến 2.30.

$$\frac{\partial L}{\partial A_t^{[2]}} = \nabla_{A_t^{[2]}} \text{Cost} \quad (\text{ví dụ: } A_t^{[2]} - Y_t) \quad (2.20)$$

$$\frac{\partial L}{\partial Z_t^{[2]}} = \frac{\partial L}{\partial A_t^{[2]}} \odot g^{[2]'}(Z_t^{[2]}) \quad (2.21)$$

$$\frac{\partial L}{\partial W^{[2]}} = \sum_{t=1}^T \frac{\partial L}{\partial Z_t^{[2]}} A_t^{[1]T} \quad (2.22)$$

$$\frac{\partial L}{\partial b^{[2]}} = \sum_{t=1}^T \sum_{\text{rows}} \frac{\partial L}{\partial Z_t^{[2]}} \quad (2.23)$$

$$\frac{\partial L}{\partial A_T^{[1]}} = W^{[2]T} \frac{\partial L}{\partial Z_T^{[2]}} \quad (2.24)$$

$$(2.25)$$

$$\frac{\partial L}{\partial A_t^{[1]}} = W^{[2]T} \frac{\partial L}{\partial Z_t^{[2]}} + W^{[1]T} \frac{\partial L}{\partial Z_{t+1}^{[1]}} \quad (\text{với } t < T) \quad (2.26)$$

$$\frac{\partial L}{\partial Z_t^{[1]}} = \frac{\partial L}{\partial A_t^{[1]}} \odot g^{[1]'}(Z_t^{[1]}) \quad (2.27)$$

$$\frac{\partial L}{\partial W^{[1]}} = \sum_{t=1}^T \frac{\partial L}{\partial Z_t^{[1]}} A_{t-1}^{[1]T} \quad (2.28)$$

$$\frac{\partial L}{\partial U^{[1]}} = \sum_{t=1}^T \frac{\partial L}{\partial Z_t^{[1]}} X_t^T \quad (2.29)$$

$$\frac{\partial L}{\partial b^{[1]}} = \sum_{t=1}^T \sum_{\text{rows}} \frac{\partial L}{\partial Z_t^{[1]}} \quad (2.30)$$

Bảng 2.1 so sánh giữa FFN + BP và RNN + BPTT một cách tổng quát.

Bảng 2.1: So sánh giữa FFN + BP và RNN + BPTT

Đặc điểm	FFN + BP thông thường	RNN + BPTT
Thời gian	Không phụ thuộc	Phụ thuộc vào chuỗi thời gian
Gradient	Truyền ngược theo layer	Truyền ngược theo layer và thời gian
Công thức	$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial Z^{[l]}} A^{[l-1]T}$	Cộng thêm gradient từ $t + 1$: $\frac{\partial L}{\partial A_t^{[1]}} =$ Output Grad + Future Hidden Grad

2.1.4 So sánh một số công thức cơ bản của Feedforward và Backpropagation trong quá trình huấn luyện mạng nơ-ron nhân tạo

Feedforward và backpropagation là hai giai đoạn cốt lõi trong quá trình huấn luyện mạng nơ-ron nhân tạo. Chúng khác nhau ở mục tiêu và cách xử lý:

- **Feedforward:** Lan truyền đầu vào qua các lớp để tính đầu ra.
- **Backpropagation:** Lan truyền ngược sai số từ đầu ra về các lớp trước đó để cập nhật trọng số.

Bảng 2.2 dưới đây so sánh công thức giữa hai giai đoạn này trong mạng nơ-ron nhiều lớp (MLP) đơn giản:

Chú thích:

- $a^{[l]}$: đầu ra của lớp thứ l
- $z^{[l]}$: giá trị trước kích hoạt tại lớp l

Bảng 2.2: So sánh giữa Feedforward và Backpropagation trong mạng nơ-ron nhiều lớp

Thành phần	Feedforward (Lan truyền xuôi)	Backpropagation (Lan truyền ngược)
Đầu vào lớp l	$a^{[l-1]}$	Sai số $\delta^{[l+1]}$ từ lớp kế tiếp
Tính toán đầu ra tuyến tính	$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$	(Không áp dụng – đã có $z^{[l]}$)
Kích hoạt phi tuyến	$a^{[l]} = \sigma(z^{[l]})$	$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \cdot \sigma'(z^{[l]})$
Sai số ở lớp đầu ra	—	$\delta^{[L]} = \nabla_a \mathcal{L} \cdot \sigma'(z^{[L]})$
Cập nhật trọng số	—	$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$
Cập nhật bias	—	$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \delta^{[l]}$

- σ : hàm kích hoạt (ví dụ: tanh, ReLU)
- $\delta^{[l]}$: sai số tại lớp l
- $\mathcal{L}$: hàm mất mát (loss function)

Bảng 2.2 thể hiện rõ rằng **feedforward** chỉ đi xuôi để tính giá trị đầu ra, còn **backpropagation** thì lan truyền ngược để điều chỉnh trọng số dựa trên sai số.

2.2 Nền tảng Toán học

2.2.1 Lan truyền ngược qua thời gian (Backpropagation Through Time - BPTT)

Đầu tiên, ta cần tính **đạo hàm của đầu ra** s_t đối với **hàm mất mát** J , được ký hiệu là:

$$\frac{\partial J}{\partial s_t} \quad (2.31)$$

Sau khi có đạo hàm này, ta sẽ **lan truyền ngược** nó qua chuỗi các trạng thái ẩn (đã được tạo ra trong quá trình lan truyền xuôi). Trong quá trình **lan truyền ngược theo thời gian** này, ta lần lượt "bóc tách" các trạng thái đã lưu và **tích lũy đạo hàm sai số** tại từng bước thời gian.

Quan hệ đệ quy để lan truyền gradient theo thời gian trong RNN - sử dụng **quy tắc đạo hàm hàm hợp (chain rule)** - được biểu diễn như sau:

$$\frac{\partial J}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot W \quad (2.32)$$

Trong đó: W là ma trận trọng số kết nối giữa trạng thái ẩn hiện tại và trạng thái ẩn trước.

Các **gradient của trọng số** U (input $\rightarrow$ hidden) và W (hidden $\rightarrow$ hidden) trong mạng RNN được **tích lũy theo toàn bộ chuỗi thời gian** như sau:

$$\frac{\partial J}{\partial U} = \sum_{t=0}^n \frac{\partial J}{\partial s_t} \cdot x_t \quad (2.33)$$

$$\frac{\partial J}{\partial W} = \sum_{t=0}^n \frac{\partial J}{\partial s_t} \cdot s_{t-1} \quad (2.34)$$

Trong đó:

- x_t là đầu vào tại thời điểm t ,
- s_t là trạng thái ẩn tại thời điểm t ,
- s_{t-1} là trạng thái ẩn tại thời điểm $t - 1$.

2.2.2 Tối ưu hóa RNN bằng Gradient Descent

Sử dụng **gradient descent** [11] để tối ưu hóa mạng RNN. Ta tính toán gradient (sử dụng Mean Squared Error - MSE) với sự trợ giúp của hàm lan truyền ngược và sử dụng chúng để cập nhật giá trị trọng số.

Quy tắc cập nhật gradient descent là:

$$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta) \quad (2.35)$$

Trong đó:

- θ : Các tham số (hoặc vector tham số) mà chúng ta đang cố gắng tối ưu hóa.
- η : Hệ số học (learning rate), một hằng số dương nhỏ.
- $J(\theta)$: Hàm mất mát (hay hàm mục tiêu) mà chúng ta muốn tối thiểu hóa.
- $\nabla_{\theta} J(\theta)$: Gradient của hàm mất mát theo θ .

Ở mỗi bước:

- Tính toán gradient $\nabla_{\theta} J(\theta)$, chỉ ra hướng tăng nhanh nhất.
- Hiệu của gradient này, được nhân với η , để di chuyển theo hướng giảm nhanh nhất (tức là, tối thiểu hóa hàm mục tiêu).

$$U = U - \eta \cdot \frac{\partial L}{\partial U} \quad (2.36)$$

$$W = W - \eta \cdot \frac{\partial L}{\partial W} \quad (2.37)$$

Trong đó:

- U và W là các ma trận trọng số trong RNN,
- $\frac{\partial L}{\partial U}$ và $\frac{\partial L}{\partial W}$ là gradient của hàm mất mát L đối với trọng số U và W tương ứng.

2.3 Các vấn đề tiềm ẩn như vanishing/exploding gradients

2.3.1 Bài toán minh họa: Đếm số lượng ký tự “1” trong chuỗi nhị phân bằng RNN

Bài toán yêu cầu xây dựng một mô hình mạng nơ-ron hồi tiếp để đếm số lượng ký tự “1” trong một dãy nhị phân đầu vào. Dãy này là một chuỗi các số 0 và 1, có thể có độ dài thay đổi. Ví dụ, với đầu vào $[1, 0, 1, 1, 0]$, mô hình cần xuất ra kết quả 3. Mô hình sẽ xử lý dữ liệu tuần tự từng phần tử, cập nhật trạng thái ẩn và học cách tích lũy số lượng ký tự “1” thông qua các bước thời gian.

Mô hình có thể là SimpleRNN [15], LSTM [12] hoặc GRU [16]. Đầu ra là một số nguyên, được huấn luyện bằng hàm mất mát như Mean Squared Error (MSE) nếu dự đoán liên tục, hoặc Cross Entropy nếu phân loại đếm số “1” theo từng lớp rời rạc.

Bài toán đơn giản này giúp minh họa khả năng của RNN trong việc ghi nhớ và xử lý thông tin theo trình tự, cũng như học các phép toán tích lũy cơ bản trên chuỗi.

Mối quan hệ hồi quy được xác định bởi mạng RNN này là:

$$s_t = f(s_{t-1}W + x_tU) \quad (2.38)$$

Để đơn giản hóa, đây là một mô hình tuyến tính và chúng ta không áp dụng hàm phi tuyến trong công thức này. Các trạng thái s_t và các trọng số W và U đều là các giá trị vô hướng, x_t đại diện cho một phần tử trong chuỗi đầu vào (có thể là 0 hoặc 1). Đặt trọng số ngõ ra $V = 1$.

- Nếu đặt $U = 1$, khi có đầu vào, ta sẽ nhận được giá trị đầy đủ của nó.
- Nếu đặt $W = 1$, giá trị ta tích lũy sẽ không bị giảm đi qua thời gian.

2.3.2 Mô tả quá trình huấn luyện RNN

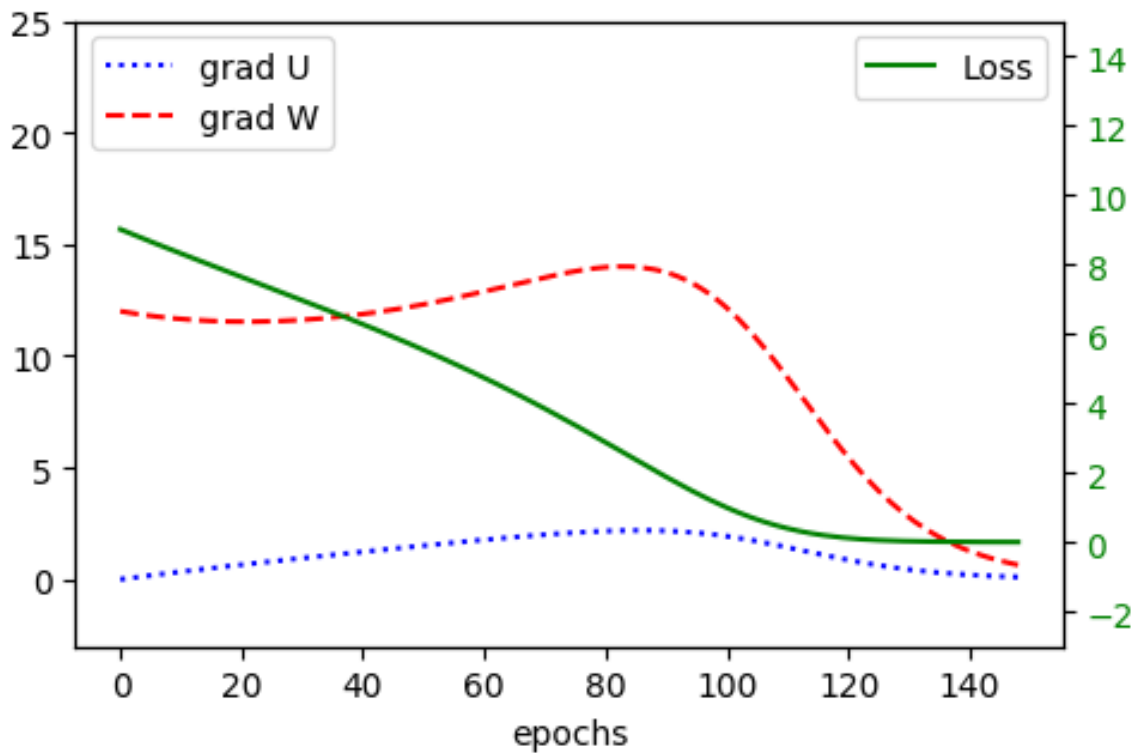
Sau khi chạy đoạn mã huấn luyện mô hình RNN với đầu vào là chuỗi nhị phân $x = [0, 0, 0, 0, 1, 0, 1, 0, 1, 0]$ và đầu ra mong đợi $y = 3$, quá trình huấn luyện diễn ra qua 150 epochs, được thể hiện trên hình 2.5, và có những thay đổi rõ rệt trong các chỉ số như sau:

- **Loss (Mất mát):** Loss giảm dần và tiệm cận 0 sau khoảng 120 epochs. Điều này cho thấy mô hình đã học được cách giảm sai số và đạt được sự hội tụ gần như hoàn hảo trong việc dự đoán đầu ra. Mô hình có thể đã học tốt mối quan hệ giữa đầu vào và đầu ra sau khoảng 120 epochs.
- **Gradient của U (Gradients of U):** Gradient của U tăng nhẹ và đạt đỉnh quanh epoch thứ 90. Điều này có thể là dấu hiệu cho thấy trong quá trình huấn luyện, mô hình cần điều chỉnh trọng số của U , sau đó dần ổn định và giảm xuống. Sau epoch thứ 90, gradient của U giảm dần và tiệm cận 0 sau epoch thứ 150, cho thấy sự ổn định trong việc cập nhật trọng số của U và mô hình đã hội tụ tốt.
- **Gradient của W (Gradients of W):** Gradient của W có sự thay đổi thú vị hơn: ban đầu giảm nhẹ, sau đó tăng và đạt đỉnh vào khoảng epoch thứ 90. Sau đỉnh, gradient của W giảm mạnh và tiệm cận 0 sau 150 epochs. Điều này có thể chỉ ra rằng mô hình cần một sự điều chỉnh đối với trọng số W ở giai đoạn giữa quá trình huấn luyện, sau đó ổn định lại khi mô hình hội tụ.
- **Tiến trình hội tụ (Convergence):** Cả hai gradients của U và W đều tiệm cận 0 sau epoch thứ 150, điều này chứng tỏ rằng các trọng số của mô hình đã được tối ưu hóa và không còn cần phải thay đổi nhiều nữa, cho thấy quá trình huấn luyện đã hoàn tất và mô hình đã hội tụ.

Qua đó cho thấy:

- Mô hình RNN đạt được sự hội tụ khá nhanh chóng sau khoảng 120 epochs với loss tiệm cận 0, cho thấy khả năng học tốt của mô hình.

- Các gradient của trọng số U và W có sự biến động đáng chú ý trong quá trình huấn luyện, đặc biệt là sự thay đổi ở epoch thứ 90, sau đó chúng giảm và ổn định, phản ánh quá trình điều chỉnh trọng số và dần ổn định khi mô hình học được đặc trưng của dữ liệu.



Hình 2.5: Kết quả huấn luyện với đầu vào gồm 3 số “1”.

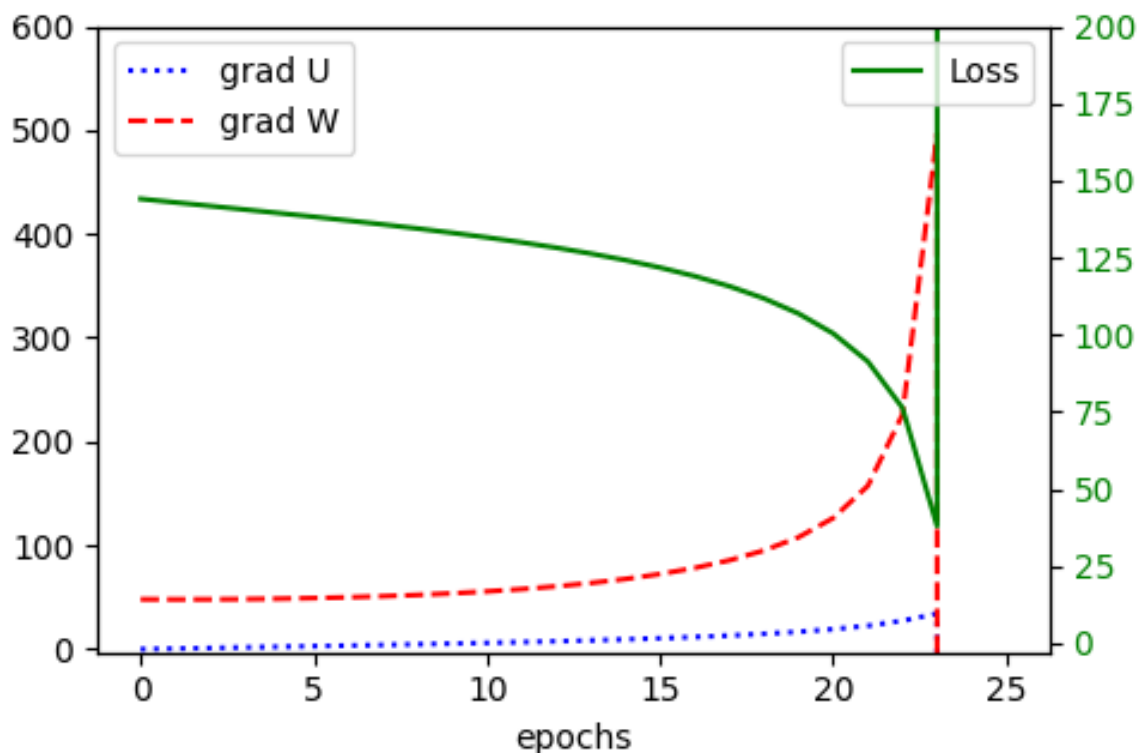
2.3.3 Mô tả hiện tượng Bùng nổ Gradient trong RNN

Sau khi thay đổi thông số huấn luyện với đầu vào là chuỗi nhị phân dài x và đầu ra mong đợi $y = 12$, quá trình huấn luyện diễn ra qua 150 epochs. Tuy nhiên, một hiện tượng đáng chú ý là sự xuất hiện của **bùng nổ gradient** sau khoảng epoch thứ 23, được thể hiện trên hình 2.6. Dưới đây là phân tích chi tiết về hiện tượng này:

- **Bùng nổ Gradient:** Trong quá trình huấn luyện, gradient của các trọng số (chẳng hạn như U và W) bắt đầu tăng lên một cách không kiểm soát sau epoch thứ 23. Điều này dẫn đến sự thay đổi mạnh mẽ trong các trọng số, gây ra hiện tượng **bùng nổ gradient**. Hiện tượng này xảy ra khi các giá trị gradient trở nên quá lớn, dẫn đến việc các trọng số bị thay đổi quá mức.

và không còn ổn định trong quá trình huấn luyện.

- **Biểu hiện của Bùng nổ Gradient:** Khi xem xét đồ thị của các gradients trong quá trình huấn luyện (được vẽ bằng hàm `plot_training`), có thể nhận thấy rằng gradients của U và W tăng đột ngột và đạt những giá trị rất lớn sau khoảng epoch thứ 23. Điều này là dấu hiệu rõ rệt của hiện tượng **bùng nổ gradient**.
- **Hậu quả của Bùng nổ Gradient:** Bùng nổ gradient có thể làm mất đi khả năng học của mô hình, vì khi trọng số thay đổi quá lớn, mô hình không thể học được các mối quan hệ trong dữ liệu một cách ổn định. Điều này có thể dẫn đến sự bất ổn trong quá trình huấn luyện, khiến mô hình không thể hội tụ vào một điểm tối ưu.
- **Cách xử lý Bùng nổ Gradient:** Một số phương pháp có thể được áp dụng để giảm thiểu hiện tượng **bùng nổ gradient** như *gradient clipping*, sử dụng các mô hình ổn định hơn (*LSTM* hoặc *GRU*), điều chỉnh *learning rate*, v.v.... Chương 3 sẽ trình bày cụ thể chi tiết từng phương pháp.



Hình 2.6: Xây ra hiện tượng exploding trong quá trình huấn luyện.

2.3.4 Phân tích nguyên nhân xảy ra Bùng nổ gradient trong mạng RNN

Bùng nổ gradient có thể xảy ra trong bất kỳ mạng nơ-ron sâu nào, nhưng chúng đặc biệt rõ rệt ở mạng nơ-ron hồi tiếp do cách mà các mô hình này xử lý các chuỗi theo thời gian.

Chiều sâu theo thời gian (Depth through Time)

- Trong mạng RNN, cùng một tập trọng số được áp dụng tại mỗi bước thời gian, điều này tạo ra một mạng sâu được *trải ra theo thời gian (unfolded through time)*.
- Với một chuỗi đầu vào dài, điều này có thể tương đương với **hàng chục hoặc hàng trăm lớp "sâu"**, tùy thuộc vào độ dài chuỗi.
- Độ sâu này làm tăng nguy cơ **bùng nổ gradient** (gradient tăng theo cấp số nhân) hoặc **triệt tiêu gradient** (gradient giảm dần về 0).

Tính toán Gradient trong RNN

$$\frac{\partial J}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot W \quad (2.39)$$

- Gradient tại một thời điểm phụ thuộc vào tích của nhiều ma trận Jacobian.
- Nếu trị riêng (eigenvalue) của các ma trận này **lớn hơn 1**, tích của chúng sẽ tăng theo cấp số nhân $\rightarrow$ **bùng nổ gradient**.

Chia sẻ tham số (Parameter Sharing)

- RNN sử dụng **cùng một tập tham số tại mỗi bước thời gian**, nên bất kỳ sự không ổn định nào cũng sẽ được **lặp lại và tích lũy**.
- Trong mạng feedforward, mỗi lớp có bộ tham số riêng biệt, nên nếu một lớp gây ra vấn đề, các lớp khác có thể điều chỉnh tham số riêng để "bù trừ". Đây là một dạng tự điều chỉnh nội bộ giữa các lớp - linh hoạt hơn trong việc xử lý nhiễu cục bộ.
- Vì RNN dùng chung một bộ tham số qua các bước thời gian, nên không có cách nào để "cân bằng lẫn nhau" giữa các bước thời gian bằng việc chỉnh tham số độc lập. Nếu bước thời gian trước gây lỗi, bước sau không thể điều chỉnh tham số riêng để sửa lỗi - nó vẫn dùng cùng một hàm. Vì vậy, so với mạng feedforward, RNN ít linh hoạt hơn theo chiều thời gian.

Phụ thuộc dài hạn (Long-Term Dependencies)

- Việc cố gắng học các mối quan hệ dài hạn khiến chiều dài lan truyền ngược (backpropagation) tăng lên, làm tăng nguy cơ bất ổn gradient.

Qua đó, có thể phần nào thấy được nguyên nhân dẫn đến hiện tượng bùng nổ gradient khi so sánh các mạng nơ-ron với nhau.

- Mạng neural feedforward thông thường có độ sâu cố định và thường nhỏ hơn.
- Mặc dù các mạng feedforward rất sâu (ví dụ: trên 100 lớp) cũng có thể gặp vấn đề gradient bùng nổ, nhưng vấn đề này **ít phổ biến hơn** so với RNN và có thể giảm thiểu với các kiến trúc hiện đại (như ResNet, các kiến trúc với các lớp chuẩn hóa).

$$\frac{\partial s_t}{\partial s_{t-k}} = \prod_{j=1}^k \phi'(z_{t-j+1})W = W^k \quad (2.40)$$

Trong một mạng **RNN tuyến tính đơn giản** với trọng số vô hướng W , gradient giữa hai thời điểm cách nhau k bước sẽ cho ra sai khác bằng một đại lượng W^k . Điều này có nghĩa là:

- Nếu $|W| > 1$ thì gradient sẽ **tăng theo cấp số nhân** khi số bước thời gian tăng lên $\rightarrow$ hiện tượng **bùng nổ gradient**.
- Nếu $|W| < 1$ thì gradient sẽ **giảm theo cấp số nhân** $\rightarrow$ hiện tượng **triệt tiêu gradient**.

Khi W là một **ma trận**, điều này được tổng quát hóa: độ lớn của gradient bị chi phối bởi **bán kính phổ** (*spectral radius*) $\rho(W)$ — tức là trị tuyệt đối lớn nhất trong các trị riêng (eigenvalue) của W :

- Nếu $\rho(W) > 1$, gradient có xu hướng **bùng nổ**.
- Nếu $\rho(W) < 1$, gradient có xu hướng **triệt tiêu**.

Tuy nhiên, hành vi cụ thể còn phụ thuộc vào **hàm kích hoạt** được sử dụng và đạo hàm của nó tại mỗi bước thời gian.

Chương 3 sẽ phân tích các giải pháp cho vấn đề bùng nổ|triệt tiêu gradient và các phương pháp huấn luyện khác.

Chương 3

Giải pháp và Các phương pháp huấn luyện khác

Vì Full BPTT có thể gây ra hiện tượng bùng nổ hoặc triệt tiêu gradient, nên Truncated BPTT được đề xuất như một giải pháp thay thế. Phương pháp này được so sánh với Full BPTT qua bảng 3.1.

3.1 Giải pháp xử lý vanishing/exploding gradients bằng Truncated BPTT

Bảng 3.1: So sánh *Full BPTT* và *Truncated BPTT*

Đặc điểm	Full BPTT	Truncated BPTT
Định nghĩa	Lan truyền ngược qua toàn bộ chuỗi	Chỉ lan truyền ngược qua một số bước cố định k
Độ dài chuỗi	Toàn bộ chuỗi (VD: 1000 bước)	Chỉ k bước gần nhất (VD: 20 bước)
Gradient flow	Nhớ được thông tin trong dài hạn	Bộ nhớ ngắn hạn, có thể bỏ lỡ thông tin dài hạn
Chi phí tính toán	Cao	Thấp hơn
Thời gian huấn luyện	Chậm hơn trên mỗi vòng lặp	Nhanh hơn
Phù hợp với	Chuỗi ngắn, cần toàn bộ ngữ cảnh	Chuỗi dài, dữ liệu thời gian thực
Bùng nổ/triệt tiêu gradients	Cao (do chuỗi dài)	Thấp hơn, nhưng vẫn tồn tại

3.1.1 Cơ sở toán học

Quy ước $w_x = 1$. Đặt:

$$h_t = f(x_t, h_{t-1}, w_h), \quad (3.1)$$

$$o_t = g(h_t, w_o), \quad (3.2)$$

Trong đó f và g lần lượt là các phép biến đổi của lớp ẩn và lớp đầu ra. Do đó, ta có một chuỗi các giá trị $\{\dots, (x_{t-1}, h_{t-1}, o_{t-1}), (x_t, h_t, o_t), \dots\}$ phụ thuộc lẫn nhau thông qua tính toán truy hồi. Lan truyền thuận khá đơn giản. Tất cả những gì cần làm là lặp qua các bộ ba (x_t, h_t, o_t) theo từng bước thời gian. Sai khác giữa đầu ra o_t và giá trị mục tiêu y_t sau đó được đánh giá bởi một hàm mục tiêu trên tất cả T bước thời gian như sau:

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t). \quad (3.3)$$

Trong công thức 2.39, nếu thay s_{t-1} bằng w_h , lúc này w_h không còn là một hằng số như đã quy ước trước đó, ta được:

$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \cdot \frac{\partial g(h_t, w_o)}{\partial h_t} \cdot \frac{\partial h_t}{\partial w_h}. \quad (3.4)$$

Hai nhân tử đầu tiên trong tích của công thức 3.4 rất dễ tính toán. Nhân tử thứ ba $\frac{\partial h_t}{\partial w_h}$ mới là phần phức tạp, vì chúng ta cần tính toán ảnh hưởng của tham số w_h lên h_t một cách truy hồi. Theo phép tính truy hồi trong 3.1, h_t phụ thuộc vào cả h_{t-1} và w_h , trong đó bản thân h_{t-1} cũng phụ thuộc vào w_h . Do đó, khi tính đạo hàm toàn phần của h_t theo w_h bằng quy tắc đạo hàm hàm hợp, ta thu được:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial w_h}. \quad (3.5)$$

Để tính gradient trên, giả sử ta có ba dãy $\{a_t\}$, $\{b_t\}$, $\{c_t\}$ thỏa mãn $a_0 = 0$ và $a_t = b_t + c_t a_{t-1}$ với $t = 1, 2, \dots$. Khi đó với $t \geq 1$, ta có thể dễ dàng chứng minh:

$$a_t = b_t + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t c_j \right) b_i. \quad (3.6)$$

Bằng cách thay thế a_t , b_t , và c_t theo các biểu thức:

$$a_t = \frac{\partial h_t}{\partial w_h}, \quad (3.7)$$

$$b_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h}, \quad (3.8)$$

$$c_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}}, \quad (3.9)$$

phép tính gradient trong 3.5 thỏa mãn $a_t = b_t + c_t a_{t-1}$. Do đó, theo 3.6, ta có thể loại bỏ tính toán truy hồi trong 3.5 bằng:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}. \quad (3.10)$$

Ta có thể sử dụng quy tắc đạo hàm hàm hợp để tính $\frac{\partial h_t}{\partial w_h}$ một cách đệ quy, chuỗi này có thể trở nên rất dài khi t lớn.

3.1.2 BPTT đầy đủ (Full BPTT)

- Sử dụng toàn bộ chuỗi để học, thể hiện qua công thức 3.10.
- Gradient được lan truyền ngược qua **tất cả các bước thời gian**.

Ưu điểm:

- Về lý thuyết, có thể học tốt hơn do ghi nhớ được thông tin dài hạn.
- Cập nhật gradient chính xác hơn.

Nhược điểm:

- **Tốn kém** cả về tính toán và bộ nhớ.
- Nguy cơ cao bị **bùng nổ hoặc triệt tiêu gradient** trong chuỗi dài.
- Không khả thi với chuỗi rất dài hoặc chuỗi thời gian (streaming).

3.1.3 BPTT cắt ngắn (Truncated BPTT) [1]

- Chuỗi được chia thành các đoạn hoặc cửa sổ có độ dài k bước thời gian.
- Lan truyền ngược chỉ được thực hiện **trong từng đoạn nhỏ**.

Ưu điểm:

- **Nhanh hơn** nhiều và tiết kiệm bộ nhớ hơn.
- Mở rộng tốt với các chuỗi **dài hoặc vô hạn** (ví dụ: chuỗi âm thanh, chuỗi văn bản).

Nhược điểm:

- Có thể **bỏ lỡ các thông tin trong dài hạn** do vượt quá kích thước của cửa sổ trượt.
- Có thể gây ra **sự phân mảnh theo thời gian** (mô hình chỉ thấy được ngữ cảnh giới hạn).

3.1.4 Ví dụ

Huấn luyện một mô hình ngôn ngữ trên một đoạn văn gồm 1000 token:

BPTT đầy đủ:

- Ghi nhận toàn bộ 1000 bước thời gian.
- Lan truyền ngược gradient qua toàn bộ đoạn văn.
- Chi phí tính toán và bộ nhớ rất cao.

BPTT cắt ngắn (ví dụ, $k = 50$):

- Xử lý từng đoạn 50 token.
- Lan truyền ngược gradient chỉ trong 50 token gần nhất.
- Nhanh hơn, nhưng có thể quên thông tin ở đầu đoạn văn.

Tổng quát hóa các tình huống sử dụng Full và Truncated BPTT được thể hiện qua bảng 3.2.

Bảng 3.2: Tổng quát hóa các tình huống sử dụng Full|Truncated BPTT

Tình huống	Phương pháp đề xuất
Tập dữ liệu nhỏ với chuỗi ngắn	BPTT đầy đủ (Full BPTT)
Chuỗi dài (ví dụ: văn bản, âm thanh)	BPTT cắt ngắn (Truncated BPTT)
Ứng dụng có dữ liệu chuỗi thời gian	BPTT cắt ngắn (Truncated BPTT)
Khi việc học các thông tin dài hạn là rất quan trọng	Cần nhắc sử dụng BPTT đầy đủ hoặc LSTM với cửa sổ cắt ngắn dài hơn

3.1.5 Truncated BPTT: Randomized Truncation vs. Standard Truncation

Phương pháp Truncated BPTT được phân thành hai chiến lược chính là Randomized Truncation [17–20] và Standard Truncation, như được minh họa trong bảng 3.3.

Bảng 3.3: So sánh *Randomized Truncation* và *Standard Truncation*

Đặc điểm	Cắt cố định (Standard)	Cắt ngẫu nhiên (Randomized)
Độ dài cắt	Cố định (VD: 20 bước)	Ngẫu nhiên theo phân phối (VD: phân phối hình học)
Tần suất cập nhật	Đều đặn	Thay đổi qua mỗi vòng
Độ biến thiên gradient	Thấp, cập nhật ổn định	Cao hơn, nhưng trung bình (kỳ vọng) tốt hơn
Học được thông tin dài hạn	Dễ bỏ lỡ do giới hạn k	Linh hoạt, có thể học thông tin dài hạn
Tính toán	Dễ đoán, hiệu quả	Ít đoán trước, khó song song hoá
Trường hợp sử dụng	Huấn luyện ổn định, chuỗi dài cố định	Học liên tục theo thời gian (online learning), môi trường stochastic

3.1.5.1 Cắt cố định (Standard Truncation)

- Chọn một **cửa sổ cố định** (ví dụ: 20 bước thời gian).
- Tại mỗi lần cập nhật, RNN xử lý một đoạn k bước và lan truyền ngược chỉ trong cửa sổ trượt đó, được thể hiện qua công thức 3.12.

Xét trường hợp ngắn nhất là 1 bước thời gian, ta có công thức đạo hàm khi áp dụng Truncation ngắn hạn ($k = 1$)

Khi áp dụng **truncation** với độ dài ngắn nhất (chỉ xét 1 bước thời gian trước đó, tức $i = t - 1$), công thức (3.10) rút gọn thành:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \underbrace{\left(\frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \right) \frac{\partial f(x_{t-1}, h_{t-2}, w_h)}{\partial w_h}}_{\text{Thành phần truncation tại } i=t-1}. \quad (3.11)$$

Giải thích:

- **Thành phần gốc:** $\frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h}$ là đạo hàm trực tiếp tại bước t .
- **Thành phần truncation:** Tổng $\sum_{i=1}^{t-1}$ trong công thức gốc được rút gọn thành một số hạng duy nhất tại $i = t - 1$.
- Tích $\prod_{j=i+1}^t$ trở thành một phần tử duy nhất là $\frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}}$.

Công thức tổng quát khi truncation sau k bước: Nếu muốn cắt ngắn gradient sau k bước, đạo hàm được xấp xỉ bởi:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=\max(1, t-k)}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}. \quad (3.12)$$

Ứng dụng:

- Đây là cơ sở của phương pháp **Truncated Backpropagation Through Time (TBPTT)**, giúp giảm chi phí tính toán bằng cách bỏ qua ảnh hưởng từ quá khứ xa.
- Trường hợp $k = 1$ là phiên bản đơn giản nhất, phù hợp với các mô hình gần-Markov hoặc khi tài nguyên tính toán hạn chế.

Ưu điểm:

- Dễ triển khai.
- Tính toán hiệu quả và ổn định.

Nhược điểm:

- Có thể tạo ra **độ chệch** trong việc ước lượng gradient.
- Có thể bỏ sót các phụ thuộc dài hoặc không đều.

3.1.5.2 Cắt ngẫu nhiên (Randomized Truncation)

- Độ dài chuỗi cắt ngắn được **lấy ngẫu nhiên** tại mỗi lần cập nhật.
- Độ dài được lấy mẫu từ một phân phối xác suất (ví dụ: phân phối hình học hoặc phân phối đều).

Thay thế $\frac{\partial h_t}{\partial w_h}$ trong công thức 3.5 bằng một biến ngẫu nhiên đúng về kỳ vọng nhưng cắt ngắn chuỗi lan truyền. Cụ thể, sử dụng một dãy ξ_t với xác suất định trước $0 \leq \pi_t \leq 1$, trong đó:

- $P(\xi_t = 0) = 1 - \pi_t$ (dừng lan truyền),
- $P(\xi_t = \pi_t^{-1}) = \pi_t$ (tiếp tục với trọng số),

đảm bảo rằng $E[\xi_t] = 1$. Khi đó, thay thế gradient $\frac{\partial h_t}{\partial w_h}$ trong công thức 3.5 bằng:

$$z_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \xi_t \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}. \quad (3.13)$$

Từ định nghĩa của ξ_t , ta có $E[z_t] = \frac{\partial h_t}{\partial w_h}$. Khi $\xi_t = 0$, quá trình lan truyền ngược dừng lại ở bước thời gian t . Kết quả là một tổng trọng số của các chuỗi có độ dài khác nhau, trong đó các chuỗi dài (hiếm) được đánh trọng số thích hợp. Ý tưởng này được đề xuất bởi Talley và Ollivier (2017) [24].

Trong công thức 3.13, ξ_t là một biến ngẫu nhiên có phân phối **Bernoulli cải biên** (modified Bernoulli distribution) hoặc **phân phối hai điểm rời rạc** (two-point discrete distribution). Cụ thể:

- ξ_t nhận giá trị 0 với xác suất $1 - \pi_t$,
- ξ_t nhận giá trị π_t^{-1} với xác suất π_t .

Kỳ vọng của ξ_t được định nghĩa là:

$$E[\xi_t] = 0 \cdot (1 - \pi_t) + \pi_t^{-1} \cdot \pi_t = 1. \quad (3.14)$$

Giải thích

1. **Phân phối Bernoulli thông thường** chỉ nhận hai giá trị 0 hoặc 1, nhưng trong trường hợp này, ξ_t nhận giá trị 0 hoặc π_t^{-1} (một giá trị khác 1) để đảm bảo kỳ vọng bằng 1.

2. Mục đích:

- Khi $\xi_t = 0$, gradient ngừng lan truyền ngược tại bước thời gian t (truncation).
- Khi $\xi_t = \pi_t^{-1}$, gradient được khuếch đại bởi π_t^{-1} để bù cho việc các chuỗi dài hiếm khi xuất hiện, đảm bảo tính không chệch (unbiased) trong tổng hợp.

Tóm tắt lại việc thực hiện cắt ngẫu nhiên

1. **Mục tiêu:** Giảm chi phí tính toán trong lan truyền ngược (backpropagation) qua các chuỗi dài bằng cách ngẫu nhiên cắt bớt một số bước.
2. **Cơ chế:**
 - Với xác suất π_t , tiếp tục lan truyền và khuếch đại gradient bằng π_t^{-1} để bù cho việc cắt ngắn.
 - Với xác suất $1 - \pi_t$, dừng lan truyền tại t .
3. **Ưu điểm:** Duy trì tính không chệch (unbiased) của gradient trong khi giảm độ phức tạp.

Phương pháp này đặc biệt hữu ích cho các mô hình RNN hoặc LSTM xử lý dữ liệu chuỗi dài.

Ví dụ:

- Thay vì luôn cắt tại 20 bước, có thể lấy độ dài cắt ngắn từ một **phân phối hình học** với kỳ vọng là 20.

Ưu điểm:

- Tạo ra một **ước lượng không chệch** của gradient BPTT đầy đủ (trong kỳ vọng).
- Cho phép mô hình **thấy được các thông tin (chuỗi dữ liệu) dài hơn**.
- Về mặt lý thuyết, thể hiện đặc tính tốt hơn khi hoạt động trên **dữ liệu online hoặc liên tục**.

Nhược điểm:

- **Phương sai cao hơn** trong ước lượng gradient $\rightarrow$ có thể làm chậm hội tụ.
- Khó song song hóa do độ dài chuỗi thay đổi.

3.1.5.3 Tóm tắt chính:

- **Cắt cố định:** hiệu quả, ổn định, nhưng có thể bị chệch (đặc biệt nếu thông tin dài hạn là quan trọng).
- **Cắt ngẫu nhiên:** cho kỳ vọng tốt hơn, giúp mô hình học được các đặc trưng đa dạng hơn, nhưng nhiều hơn.

Tổng quát hóa các tình huống sử dụng Randomized và Standard Truncation được thể hiện qua bảng 3.4.

Bảng 3.4: Tổng quát hóa các tình huống sử dụng Randomized|Standard Truncation

Tình huống	Phương pháp tốt nhất
Huấn luyện nhanh, ổn định trên chuỗi dài	Cắt cố định
Xấp xỉ BPTT đầy đủ mà không tốn nhiều chi phí, hoặc cải thiện khả năng khái quát hóa	Cắt ngẫu nhiên
Học tăng cường, học liên tục theo thời gian (online learning) hoặc trong môi trường ngẫu nhiên	Thường ưu tiên cắt ngẫu nhiên

3.2 Các phương pháp huấn luyện khác

Các hướng tiếp cận khác bao gồm:

- Gradient Clipping [21]
- BPTT trong LSTM và GRU
- Phương pháp không dùng gradient (Gradient-free methods) [22]
- Real-Time Recurrent Learning (RTRL) [23]

3.2.1 Gradient Clipping

Tác dụng:

- Gradient clipping **giới hạn (cắt)** độ lớn của gradient trong quá trình lan truyền ngược.
- Nếu chuẩn của gradient vượt quá một ngưỡng nhất định, nó sẽ được **chuẩn hóa lại** để tránh tăng quá lớn.

Lợi ích:

- Ngăn chặn gradient bị **bùng nổ**, tránh gây ra **bất ổn định số học** hoặc làm mô hình **hội tụ sai**.
- Đặc biệt hữu ích trong **RNN tiêu chuẩn** khi huấn luyện trên chuỗi dài.

Biểu thức toán học:

Nếu chuẩn gradient $\|g\|$ vượt quá ngưỡng τ , ta chuẩn hóa lại như sau:

$$g \leftarrow \frac{\tau}{\|g\|} \cdot g$$

Điều này giúp cập nhật ổn định mà không làm gián đoạn quá trình học.

3.2.2 BPTT trong LSTM và GRU

LSTM và **GRU** được thiết kế để vượt qua các vấn đề gradient bằng cách sử dụng các **cơ chế cổng (gating)** nhằm kiểm soát dòng thông tin theo thời gian. Các cổng này giúp mô hình duy trì gradient quan trọng và kiểm soát việc "quên" hay "ghi nhớ" thông tin tại mỗi bước thời gian.

3.2.2.1 LSTM (Long Short-Term Memory)

LSTM có cấu trúc phức tạp hơn với **ba cổng chính**:

- **Cổng quên (Forget gate)**: Quyết định loại bỏ thông tin nào khỏi trạng thái ô nhớ.
- **Cổng đầu vào (Input gate)**: Xác định thông tin mới nào sẽ được lưu vào ô nhớ.
- **Cổng đầu ra (Output gate)**: Điều khiển đầu ra dựa trên đầu vào hiện tại và trạng thái ẩn trước đó.

BPTT trong LSTM

BPTT trong LSTM hoạt động như sau:

1. Trạng thái cell và luồng gradient:

- Trạng thái ô (*cell state*) được truyền qua thời gian gần như không thay đổi, cho phép gradient lan truyền qua nhiều bước mà không bị triệt tiêu hay bùng nổ.
- Các cổng giúp mô hình chọn lọc **bỏ hoặc thêm thông tin** vào trạng thái ô tại mỗi bước thời gian.

2. Tính toán gradient:

- Trong quá trình BPTT, gradient của hàm mất mát được lan truyền ngược qua thời gian, nhưng trạng thái ô giúp **giữ lại gradient** qua nhiều bước.
- Gradient được **nhân và điều chỉnh** bởi các cổng quên và đầu vào, giúp giữ tính ổn định.
- Các cổng tạo ra các phép tính tích (multiplicative paths) có kiểm soát, đảm bảo gradient không bị triệt tiêu hay bùng nổ.

Mathematical Formulation:

- Gradient của hàm mất mát đối với trọng số tại mỗi bước thời gian bị ảnh hưởng bởi các **cổng (gates)**. Ví dụ, bản cập nhật cho trạng thái ô nhớ sẽ phụ thuộc vào cách các **cổng quên** và **cổng đầu vào** tương tác với các gradient đầu vào:

$$\frac{\partial L}{\partial C_t} = \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial C_t}$$

Trong đó, C_t là trạng thái ô, và thuật ngữ gradient bị điều chỉnh bởi các cổng kiểm soát luồng của nó.

3.2.2.2 GRU (Gated Recurrent Unit)

GRU là một phiên bản đơn giản hơn của LSTM, với **hai cổng**:

- **Cổng cập nhật (Update gate)**: Quyết định bao nhiêu phần của bộ nhớ trước đó được duy trì.
- **Cổng đặt lại (Reset gate)**: Quyết định bao nhiêu phần bộ nhớ trước đó cần được quên đi khi cập nhật trạng thái ẩn.

BPTT trong GRU:**1. Cấu trúc cổng đơn giản hơn:**

- Không giống như LSTM có các cổng đầu vào và quên riêng biệt, GRU kết hợp chúng lại thành một cổng **cập nhật (update gate)** duy nhất.
- Cổng **đặt lại (reset gate)** giúp quyết định bao nhiêu phần bộ nhớ trước đó sẽ được sử dụng khi tính toán trạng thái ẩn hiện tại.

2. Luồng gradient:

- Các gradient được truyền qua cổng cập nhật và cổng đặt lại, giúp kiểm soát việc thông tin được giữ lại hay quên đi, ổn định luồng gradient qua các bước thời gian.

3. Tính toán gradient:

- Luồng gradient qua GRU được kiểm soát bởi các cổng, tương tự như trong LSTM nhưng ở dạng đơn giản hơn.
- Cổng cập nhật đảm bảo rằng mô hình có thể **duy trì** hoặc **cập nhật** bộ nhớ của mình mà không để gradient bị triệt tiêu hay bùng nổ.

So sánh luồng gradient trong BPTT của các mô hình được thể hiện qua bảng 3.5.

Bảng 3.5: So sánh luồng gradient trong BPTT của các mô hình

Kiến trúc	Luồng gradient theo thời gian	Đặc điểm chính của BPTT
RNN thường	Gradient được truyền qua cùng một ma trận trọng số lặp lại, dễ gây ra hiện tượng gradient triệt tiêu hoặc bùng nổ	Không có cơ chế kiểm soát bộ nhớ, dẫn đến sự mất ổn định
LSTM	Trạng thái ô nhớ giúp duy trì gradient qua thời gian; các cổng điều tiết dòng thông tin, ngăn gradient bị triệt tiêu/bùng nổ	Các cổng (đầu vào, quên, đầu ra) kiểm soát luồng gradient và lưu trữ bộ nhớ
GRU	Cổng cập nhật và đặt lại kiểm soát bộ nhớ và dòng thông tin, giúp giảm vấn đề gradient	Cấu trúc cổng đơn giản hơn LSTM nhưng vẫn kiểm soát luồng gradient hiệu quả

Tóm tắt cách LSTM và GRU giảm thiểu vấn đề gradient:

- **LSTM** sử dụng **trạng thái ô nhớ** để cho phép gradient truyền qua nhiều bước thời gian mà không bị triệt tiêu hoặc bùng nổ. Các cổng (quên, đầu vào, đầu ra) cho phép điều tiết thông tin được ghi nhớ và truyền đi.
- **GRU** dù đơn giản hơn, sử dụng **cổng cập nhật và đặt lại** để kiểm soát dòng thông tin và đảm bảo gradient ổn định khi lan truyền ngược qua thời gian.
- Trong cả hai kiến trúc, **cơ chế cổng** là chìa khóa giúp mạng học được các thông tin dài hạn mà không gặp phải các vấn đề gradient thường thấy ở RNN thường.

3.2.3 Phương pháp không dùng gradient (Gradient-free methods)

Phương pháp không dùng gradient là các kỹ thuật tối ưu **không dựa vào thông tin đạo hàm** để cập nhật tham số mô hình. Thay vì tính gradient (như trong gradient descent), chúng sử dụng các chiến lược khác như:

- **Lấy mẫu ngẫu nhiên (random sampling)**
- **Thuật toán tiến hóa (evolutionary algorithms)**
- **Tìm kiếm theo mẫu (pattern-based search)**

Khi nào nên dùng:

- Khi **gradient khó tính toán**, bị nhiễu, hoặc không xác định (ví dụ: hàm rời rạc, không khả vi, hoặc dạng hộp đen).
- Khi **chi phí hàm đánh giá cao** hoặc không thể tính đạo hàm giải tích.
- Trong các bài toán như **học tăng cường, mô hình mô phỏng**, hoặc **tối ưu tổ hợp**.

Các phương pháp tối ưu phổ biến không dùng gradient được tóm tắt qua bảng 3.6.

Bảng 3.6: Đánh giá các phương pháp tối ưu phổ biến không dùng gradient

Phương pháp	Mô tả
Random Search	Thử ngẫu nhiên các cấu hình tham số và chọn ra cấu hình tốt nhất. Đơn giản nhưng kém hiệu quả.
Grid Search	Khám phá có hệ thống một lưới tham số. Hiệu quả chỉ khi số chiều thấp.
Thuật toán tiến hóa (Evolutionary Algorithms)	Lấy cảm hứng từ chọn lọc tự nhiên (ví dụ: Genetic Algorithms, Covariance Matrix Adaptation Evolution Strategy (CMA-ES)). Phát triển một quần thể nghiệm.
Tối ưu Bayes (Bayesian Optimization)	Xây dựng mô hình xác suất của hàm mục tiêu và chọn điểm đánh giá dựa trên hàm lợi ích.
Simulated Annealing	Tìm kiếm ngẫu nhiên với “nhiệt độ” giảm dần để điều khiển khả năng chấp nhận nghiệm kém hơn.
Particle Swarm Optimization (PSO)	Phương pháp dựa trên quần thể, nơi các hạt di chuyển trong không gian tìm kiếm, chịu ảnh hưởng bởi nghiệm tốt nhất của chính chúng và của hàng xóm lân cận.

Ứng dụng:

- **Tối ưu siêu tham số** (ví dụ: trong AutoML hoặc học sâu).
- **Học tăng cường** (ví dụ: tìm kiếm chính sách hộp đen).
- **Robot học** (ví dụ: bộ điều khiển vật lý không thể tính gradient).
- **Tìm kiếm kiến trúc mạng** và tối ưu các hàm mục tiêu không khả vi.

Hạn chế:

- Thường **chậm hơn** và **kém hiệu quả** khi so với các phương pháp dựa trên gradient (khi hàm trơn và khả vi).
- **Không mở rộng tốt** trong không gian tham số có nhiều chiều nếu không được điều chỉnh phù hợp.

Các phương pháp không dùng gradient là công cụ mạnh mẽ khi gradient **không khả dụng**, **không đáng tin cậy** hoặc **quá tốn kém để tính toán**. Dù có thể chậm hơn, chúng lại **linh hoạt hơn** và áp dụng được trong nhiều bài toán thực tế mà phương pháp dùng gradient không thể áp dụng.

3.2.4 Real-Time Recurrent Learning (RTRL)

BPTT so với RTRL được thể hiện qua bảng 3.7.

Bảng 3.7: Tổng quan: BPTT so với RTRL

Đặc điểm	BPTT	RTRL
Phân loại	Học offline / theo lô (batch)	Học liên tục theo thời gian (online learning) / thời gian thực
Tính toán gradient	Mở rộng mạng theo thời gian rồi lan truyền ngược	Tính gradient đệ quy theo thời gian thực
Bộ nhớ sử dụng	Cao (cần lưu toàn bộ chuỗi để lan truyền ngược)	Rất cao (phải lưu toàn bộ đạo hàm riêng từng phần)
Độ phức tạp tính toán	$O(T \cdot n^2)$	$O(n^4)$ (rất tốn kém)
Phù hợp với chuỗi dài	Có (có thể cắt ngắn)	Không (quá tốn tài nguyên)
Dữ liệu luồng / thời gian thực	Không phù hợp (cập nhật chậm)	Rất phù hợp (cập nhật từng bước thời gian)
Khả năng song song hóa	Dễ dàng (phù hợp với xử lý theo lô)	Khó (cập nhật tuần tự)

Có thể thấy rằng:

- RTRL là một thuật toán **học theo thời gian thực (online)** — cập nhật trọng số **ngay sau mỗi bước thời gian**.
- Duy trì đầy đủ **ma trận Jacobian** biểu diễn mức độ phụ thuộc của trạng thái ẩn vào từng trọng số.
- Tại mỗi bước thời gian, nó cập nhật cả trạng thái ẩn và gradient của trạng thái đó đối với các trọng số.

Ưu điểm:

- **Học thực sự theo thời gian thực** — lý tưởng cho dữ liệu luồng hoặc hệ thống điều khiển.
- **Không cần cắt ngắn** — gradient được lan truyền đầy đủ.

Nhược điểm:

- **Không khả thi về mặt tính toán** đối với mạng lớn: độ phức tạp $O(n^4)$.

- Khó mở rộng hoặc triển khai trong thực tế.

Ngoài ra, các điểm khác biệt chính giữa BPTT và RTRL được thể hiện qua bảng 3.8.

Bảng 3.8: Các điểm khác biệt chính giữa BPTT và RTRL

Khía cạnh	BPTT	RTRL
Thời gian cập nhật	Cập nhật sau mỗi chuỗi hoặc đoạn (chunk)	Cập nhật ngay tại mỗi bước thời gian
Độ chính xác của gradient	Cắt ngắn, theo batch, hoặc đầy đủ	Chính xác và theo thời gian thực
Hiệu quả tính toán	Hiệu quả nếu dùng cắt ngắn	Không hiệu quả với mô hình lớn
Tính ứng dụng thực tế	Rất phổ biến trong học sâu	Hiếm gặp, chủ yếu trong nghiên cứu

Việc lựa chọn sử dụng BPTT và RTRL được thể hiện qua bảng 3.9.

Bảng 3.9: Lựa chọn sử dụng BPTT và RTRL

Tình huống	Phương pháp khuyến nghị
Huấn luyện RNN trên tập dữ liệu lớn	BPTT (có thể cắt ngắn)
Điều khiển thời gian thực hoặc robot học	RTRL (hoặc các biến thể gần đúng như Unbiased Online Recurrent Optimization (UORO))
Dữ liệu luồng hoặc học suốt đời (lifelong learning)	Biến thể của RTRL (nếu tài nguyên tính toán cho phép)

Vì RTRL có độ phức tạp tính toán quá cao nên một số **phương pháp xấp xỉ RTRL thời gian thực** đã được đề xuất nhằm duy trì khả năng cập nhật thời gian thực nhưng giảm chi phí tính toán:

- **UORO** (Unbiased Online Recurrent Optimization) [24]
- **KF-RTRL** (Kronecker-Factored RTRL) [25]
- **NoBackTrack** (Bộ ước lượng gradient xấp xỉ) [26]

Những phương pháp này hướng tới việc **giữ lại khả năng huấn luyện thời gian thực** nhưng với chi phí tính toán hợp lý hơn.

Chương 4 sẽ trình bày các ứng dụng của BPTT.

Chương 4

Các ứng dụng của BPTT

4.1 Các ứng dụng trong các lĩnh vực khác nhau

4.1.1 Xử lý ngôn ngữ tự nhiên (NLP)

BPTT là công cụ mạnh mẽ trong việc huấn luyện các mô hình học sâu, đặc biệt là trong các ứng dụng xử lý ngôn ngữ tự nhiên (NLP) như:

- **Mô hình ngôn ngữ (Language Modeling):** Dự đoán xác suất của từ tiếp theo trong một chuỗi từ, hỗ trợ nhiều ứng dụng như tìm kiếm văn bản, tự động điền câu, và phân tích cảm xúc.
- **Dịch máy (Machine Translation):** Sử dụng RNN để dịch một câu từ ngôn ngữ này sang ngôn ngữ khác, với BPTT đảm bảo việc lan truyền lỗi qua các từ trong câu dịch.
- **Nhận dạng giọng nói (Speech Recognition):** Dùng BPTT để học và nhận dạng các chuỗi tín hiệu âm thanh, chuyển đổi chúng thành văn bản. Điều này đặc biệt hữu ích trong các ứng dụng như trợ lý ảo và hệ thống nhận diện giọng nói.

4.1.2 Dự báo chuỗi thời gian

BPTT là phương pháp quan trọng trong việc học từ dữ liệu chuỗi thời gian. Một số ứng dụng phổ biến của BPTT trong dự báo chuỗi thời gian bao gồm:

- **Dự báo giá cổ phiếu (Stock Price Forecasting):** Sử dụng RNN và BPTT để phân tích dữ liệu lịch sử và dự báo biến động giá cổ phiếu trong tương lai.

- **Dự báo thời tiết (Weather Prediction):** RNN có thể học các mẫu thời gian từ dữ liệu thời tiết quá khứ để dự đoán các điều kiện khí hậu trong tương lai.

4.1.3 Robotics

Trong lĩnh vực robotics, BPTT đóng vai trò quan trọng trong việc huấn luyện các mô hình điều khiển động cơ và dự đoán chuỗi hành động của robot:

- **Dự đoán chuỗi hành động (Sequence Prediction):** BPTT có thể giúp robot dự đoán chuỗi hành động cần thiết để hoàn thành một nhiệm vụ cụ thể.
- **Điều khiển động cơ (Motor Control):** Sử dụng BPTT để học cách điều khiển các động cơ trong các robot, chẳng hạn như việc điều khiển cánh tay robot trong các tác vụ tinh vi.

4.1.4 Học tăng cường (Reinforcement Learning)

Trong học tăng cường, các mô hình học sâu với BPTT có thể học từ các tín hiệu "thưởng" liên tiếp để tối ưu hóa hành động:

- **Học từ tín hiệu "thưởng" chuỗi (Learning from Sequential Reward Signals):** BPTT giúp mô hình học từ các quyết định trong quá khứ ảnh hưởng đến các tín hiệu thưởng nhận được sau đó. Ví dụ, trong các trò chơi video, các hệ thống học tăng cường sử dụng BPTT để tối ưu hóa các chiến lược thắng cuộc.

Kết luận

BPTT đã chứng tỏ là một công cụ vô giá trong việc giải quyết các bài toán phức tạp trong nhiều lĩnh vực khác nhau. Từ xử lý ngôn ngữ tự nhiên, dự báo chuỗi thời gian, đến điều khiển robot và học tăng cường, các ứng dụng của BPTT đã là nền tảng làm thay đổi cách chúng ta tiếp cận và giải quyết các vấn đề học máy.

Chương 5 sẽ trình bày các hạn chế, xu hướng nghiên cứu và định hướng tương lai.

Chương 5

Hạn chế, Xu hướng nghiên cứu và Định hướng tương lai

5.1 Những hạn chế hiện tại của RNN và BPTT

Mặc dù Mạng nơ-ron hồi tiếp (RNN) cùng với thuật toán **Backpropagation Through Time (BPTT)** đã mở ra khả năng xử lý chuỗi dữ liệu hiệu quả, vẫn còn tồn tại nhiều hạn chế đáng kể:

- **Chi phí tính toán cao:** BPTT yêu cầu phải *mở rộng RNN theo thời gian*, dẫn đến tăng mạnh kích thước đồ thị tính toán, gây tốn thời gian và tài nguyên.
- **Tiêu tốn bộ nhớ:** Việc lưu trữ các trạng thái ẩn và trung gian tại từng bước thời gian trong quá trình lan truyền thuận (forward) để sử dụng trong lan truyền ngược (backward) là rất tốn bộ nhớ, đặc biệt với chuỗi dài.
- **Vấn đề gradient:** Như đã phân tích, BPTT thường gặp hiện tượng *vanishing* và *exploding gradient*, làm cho mô hình học kém hiệu quả khi cần ghi nhớ các phụ thuộc dài hạn.
- **Khó song song hóa:** Do tính chất phụ thuộc liên thời gian, các bước trong BPTT không thể thực hiện song song, gây khó khăn trong tối ưu và huấn luyện trên phần cứng hiện đại.

5.2 Các hướng tiếp cận thay thế: Cơ chế Attention và Transformer

Để vượt qua các giới hạn của RNN và BPTT, nhiều nghiên cứu gần đây đã chuyển sang sử dụng các mô hình phi tuần tự như:

- **Attention Mechanism** [27]: Cơ chế cho phép mô hình tập trung vào các phần thông tin quan trọng trong chuỗi đầu vào mà không cần duy trì trạng thái tuần tự, giải quyết triệt để vấn đề ghi nhớ dài hạn.
- **Transformer** [27]: Kiến trúc này hoàn toàn loại bỏ RNN, thay vào đó dùng (*multi-head attention*) và lan truyền thông tin song song. Transformer hiện đang là nền tảng của nhiều mô hình mạnh như BERT, GPT, T5, v.v.

5.3 Xu hướng nghiên cứu gần đây

Các nhà nghiên cứu đang tìm cách kết hợp BPTT truyền thống với các công nghệ mới để cải thiện hiệu quả:

- **BPTT + Memory Networks** [28]: Mạng bộ nhớ ngoài (*external memory networks*) cho phép mô hình ghi nhớ thông tin quan trọng mà không cần truyền toàn bộ gradient qua chuỗi thời gian.
- **Spiking Neural Networks (SNN)** [29]: Mô hình nơ-ron sinh học sử dụng xung (*spike*) và lan truyền thời gian thực. Đây là hướng tiềm năng cho các ứng dụng thời gian thực và tiết kiệm năng lượng.
- **Gradient Approximation** [30]: Các kỹ thuật ước lượng gradient (như REINFORCE, e-prop) giúp thay thế BPTT trong các hệ thống có yêu cầu tính toán thấp hơn.

5.3.1 Định hướng tương lai

Một số hướng đi đang được cộng đồng quan tâm để cải tiến BPTT và RNN:

- **Các biến thể BPTT hiệu quả hơn** đã được trình bày trong chương 3: Truncated BPTT, Randomized Truncation và Online BPTT đang được nghiên cứu để giảm độ phức tạp mà vẫn giữ được khả năng học phụ thuộc dài hạn.

- **Neuromorphic Computing** [31]: Kết hợp mạng nơ-ron và phần cứng đặc biệt (như Loihi của Intel) để chạy các mô hình RNN/SNN (Spiking Neural Network) với hiệu suất và năng lượng cao hơn.
- **Hybrid Architectures** [32–37]: Mô hình kết hợp giữa RNN và Transformer, hoặc giữa cơ chế attention và memory, để khai thác ưu điểm của cả hai.
- **Continual Learning**: Phát triển các mô hình có khả năng học liên tục theo thời gian, mà không bị quên các kiến thức cũ (catastrophic forgetting) [38, 39].

Kết luận

Mặc dù BPTT là một nền tảng quan trọng trong học sâu cho dữ liệu tuần tự, những giới hạn của nó đang mở đường cho các xu hướng nghiên cứu mới. Việc cải tiến thuật toán, áp dụng phần cứng mới, và chuyển hướng sang mô hình Attention đang làm thay đổi cách chúng ta xử lý chuỗi thời gian trong học máy hiện đại.

Chương 6 là phụ lục trình bày mở rộng cho các kiến thức bổ trợ liên quan.

Chương 6

Phụ lục

6.1 Perceptron đơn (Single Perceptron):

- Một **perceptron** là loại mạng nơ-ron đơn giản nhất, chỉ bao gồm **một lớp duy nhất** (không có lớp ẩn).
- Nó chỉ có thể giải các bài toán **phân loại tuyến tính**, tức là khi các lớp có thể được phân tách bằng một **đường thẳng** (trong không gian 2 chiều). Điều này được gọi là **ranh giới quyết định tuyến tính** (linear decision boundary).
- **Ví dụ về bài toán tuyến tính:** các cổng logic **AND**, **OR**.

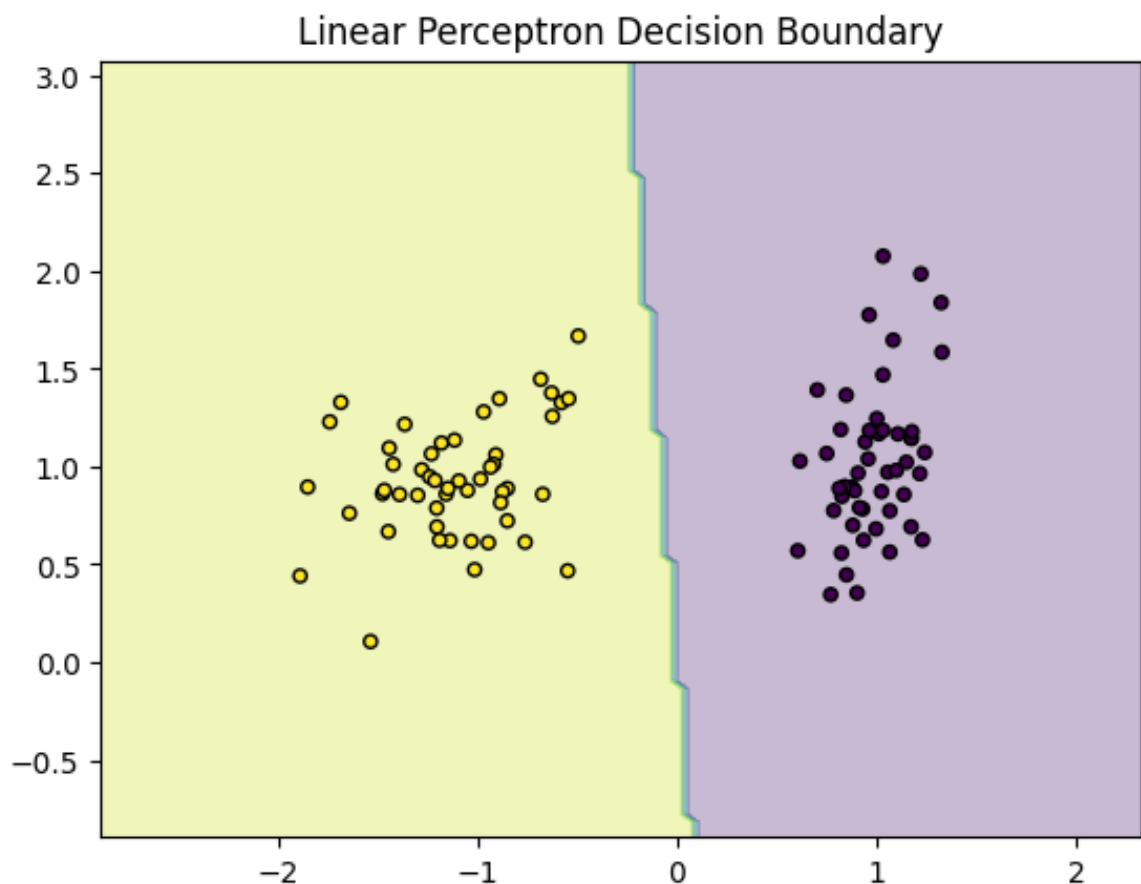
Hình 6.1 thể hiện ranh giới quyết định của Perceptron tuyến tính.

6.2 Mạng nơ-ron nhiều lớp (Multilayer Neural Network) [2]

- Các mạng này có **các lớp ẩn**, cho phép chúng **kết hợp đặc trưng** theo cách phức tạp hơn.
- Chúng có thể giải quyết các bài toán **phi tuyến** — nghĩa là bề mặt phân tách (decision surface) có thể **cong hoặc xoắn phức tạp**.
- **Ví dụ về bài toán phi tuyến:** cổng logic **XOR**.

Ngoài ra,

- Một **perceptron đơn lớp** chỉ có thể phân tách dữ liệu bằng một **đường thẳng** (tuyến tính).



Hình 6.1: Ranh giới quyết định của Perceptron tuyến tính.

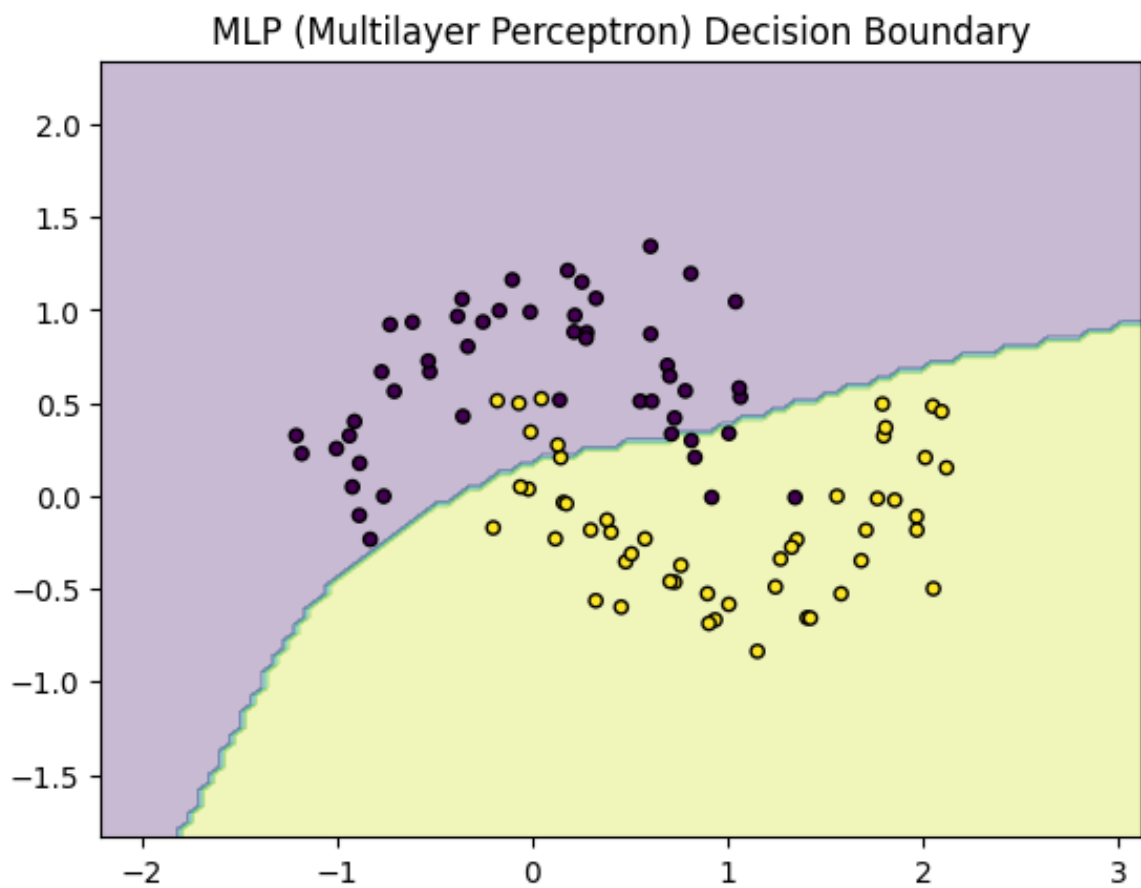
- Một **mạng nhiều lớp** có thể mô hình hóa các **ranh giới quyết định phi tuyến** nhờ vào các lớp ẩn và thuật toán lan truyền ngược được thể hiện qua hình 6.3.
- Vì lý do đó, mạng nhiều lớp là **thiết yếu để giải các bài toán phức tạp trong thế giới thực**.

Hình 6.2 thể hiện ranh giới quyết định của Perceptron nhiều lớp.

6.3 Incremental Gradient Descent và Các biến thể

6.3.1 Batch Gradient Descent [2]

- **Mô tả:** Sử dụng toàn bộ dữ liệu huấn luyện để tính gradient và cập nhật trọng số mỗi lần.
- **Ưu điểm:** Ổn định hơn, hướng gradient chính xác.



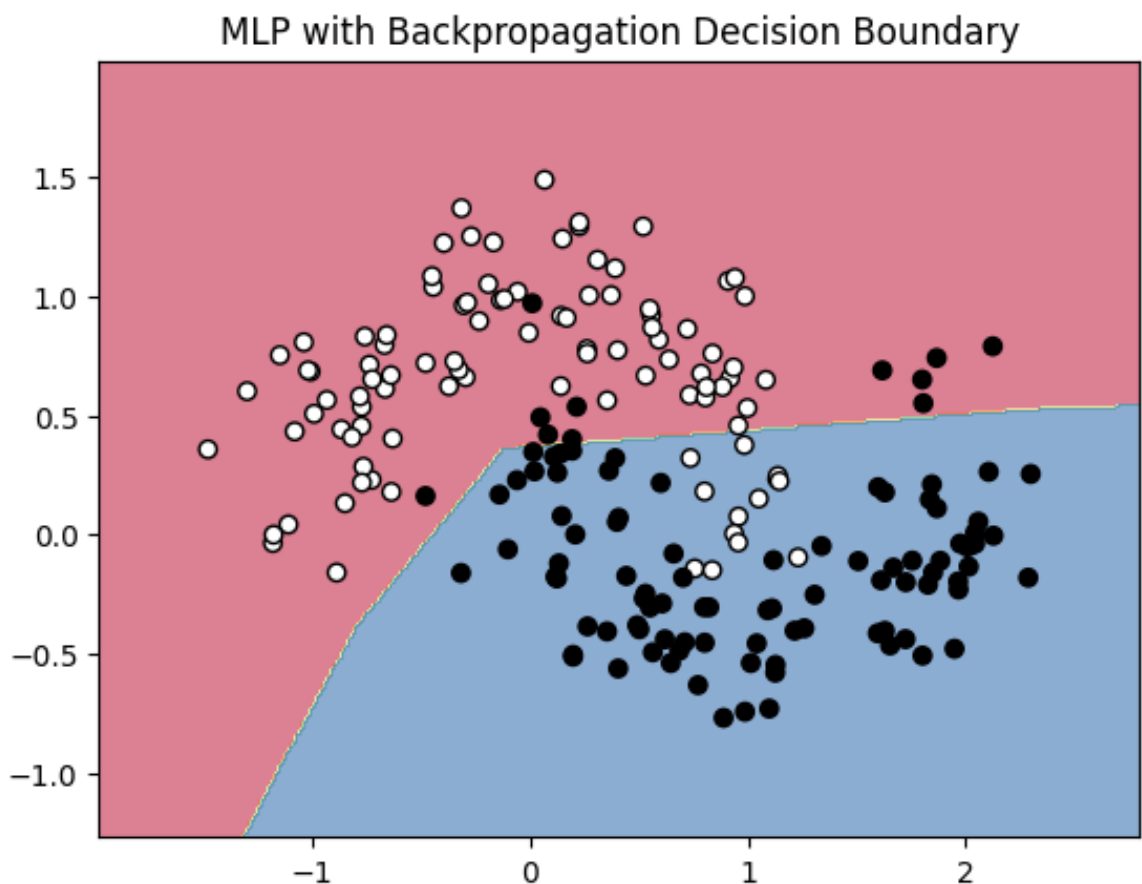
Hình 6.2: Ranh giới quyết định của Perceptron nhiều lớp.

- **Nhược điểm:** Tốn bộ nhớ và thời gian nếu tập dữ liệu lớn.

Ngoài **incremental gradient descent** (hay còn gọi là **stochastic gradient descent - SGD**) trong thuật toán backpropagation, còn có nhiều biến thể và phương pháp tối ưu khác được sử dụng rộng rãi:

6.3.2 Mini-Batch Gradient Descent [2]

- **Mô tả:** Chia dữ liệu thành các lô nhỏ và cập nhật trọng số theo từng mini-batch.
- **Ưu điểm:** Cân bằng giữa tốc độ và độ ổn định. Là phương pháp phổ biến nhất hiện nay.
- **Nhược điểm:** Cần chọn kích thước batch hợp lý.



Hình 6.3: Ranh giới quyết định của Perceptron nhiều lớp có sử dụng thuật toán lan truyền ngược.

6.3.3 Momentum [3]

- **Mô tả:** Thêm thành phần "quán tính" vào gradient để giảm dao động và tăng tốc hội tụ.
- **Ưu điểm:** Giúp vượt qua điểm cực tiểu cục bộ hoặc vùng phẳng.
- **Ghi chú:** Đây là kỹ thuật cải tiến, không phải là một loại gradient descent riêng biệt.

6.3.4 Nesterov Accelerated Gradient (NAG) [2]

- **Mô tả:** Giống Momentum nhưng tính toán gradient tại vị trí "nhìn trước".
- **Ưu điểm:** Giảm khả năng vượt quá đích, hội tụ nhanh và ổn định hơn Momentum.

6.3.5 Adagrad [4]

- **Mô tả:** Điều chỉnh tốc độ học theo từng tham số, giảm learning rate theo thời gian.
- **Ưu điểm:** Phù hợp với dữ liệu thưa (sparse).
- **Nhược điểm:** Learning rate giảm quá nhanh $\rightarrow$ dễ ngưng học sớm.

6.3.6 RMSProp [5]

- **Mô tả:** Biến thể của Adagrad, dùng trung bình động để điều chỉnh learning rate.
- **Ưu điểm:** Giữ được learning rate hiệu quả lâu dài, phù hợp với RNN.

6.3.7 Adam (Adaptive Moment Estimation) [4]

- **Mô tả:** Kết hợp giữa Momentum và RMSProp.
- **Ưu điểm:** Tự động điều chỉnh tốc độ học, thường là lựa chọn mặc định trong deep learning.
- **Nhược điểm:** Có thể không hội tụ về nghiệm tối ưu toàn cục trong một số trường hợp.

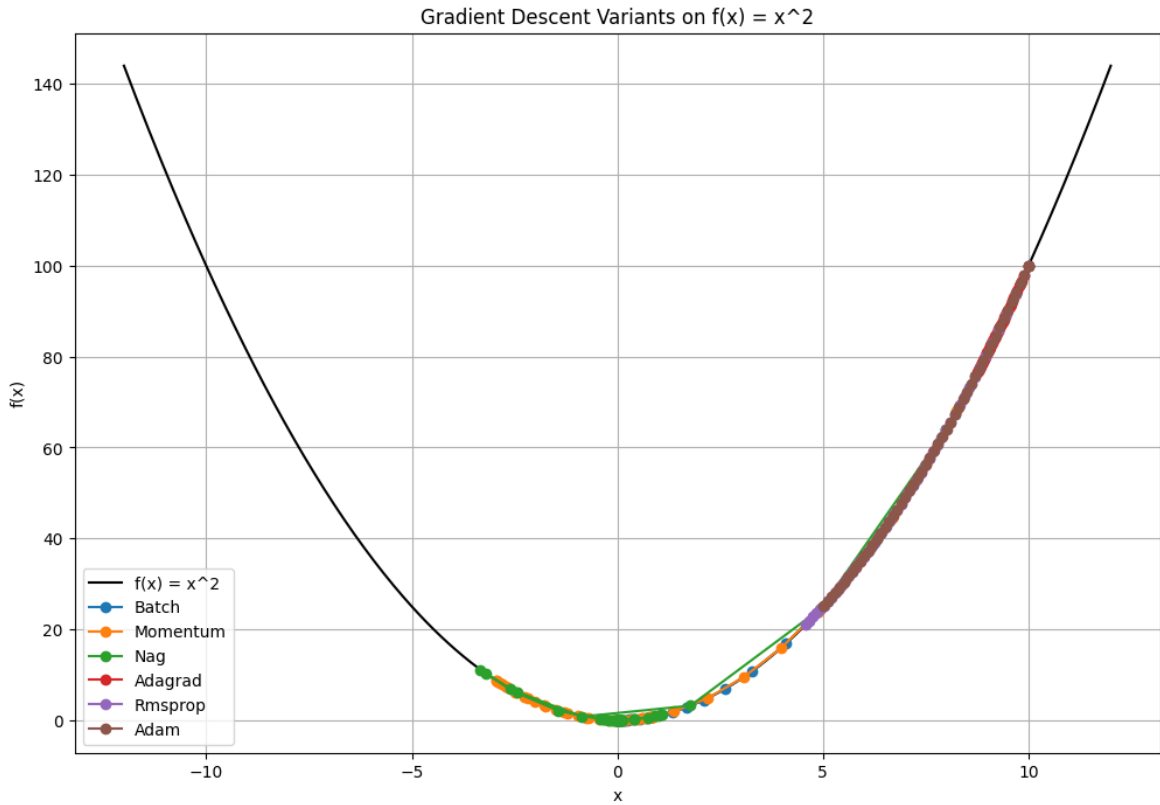
6.3.8 Các biến thể khác: AdaDelta / Nadam / AMSGrad [6]

- Đây là các thuật toán mở rộng hoặc cải tiến từ các phương pháp nêu trên.
- Mỗi thuật toán có ưu điểm riêng, thích hợp cho các tình huống và mô hình cụ thể.

Hình 6.4 thể hiện kết quả so sánh gradient descent variants đối với hàm bậc 2.

6.4 Thuật toán Backpropagation

1. Khởi tạo tất cả các trọng số w_{ji} với các giá trị ngẫu nhiên nhỏ.
2. Lặp lại cho đến khi hội tụ (hoặc trong một số vòng lặp cố định):



Hình 6.4: So sánh gradient descent variants đối với hàm bậc 2.

(a) Với mỗi mẫu huấn luyện $(\mathbf{x}, \mathbf{t})$:

i. **Lan truyền tiến (Forward Pass)**

Đưa đầu vào $\mathbf{x}$ vào mạng và tính toán đầu ra o_k tại các node đầu ra.

ii. **Tính lỗi tại lớp đầu ra**

Với mỗi node đầu ra k :

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \quad (6.1)$$

iii. **Tính lỗi tại lớp ẩn**

Với mỗi node ẩn h :

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in \text{outputs}} w_{kh} \delta_k \quad (6.2)$$

iv. **Cập nhật trọng số**

Với mỗi trọng số w_{ji} :

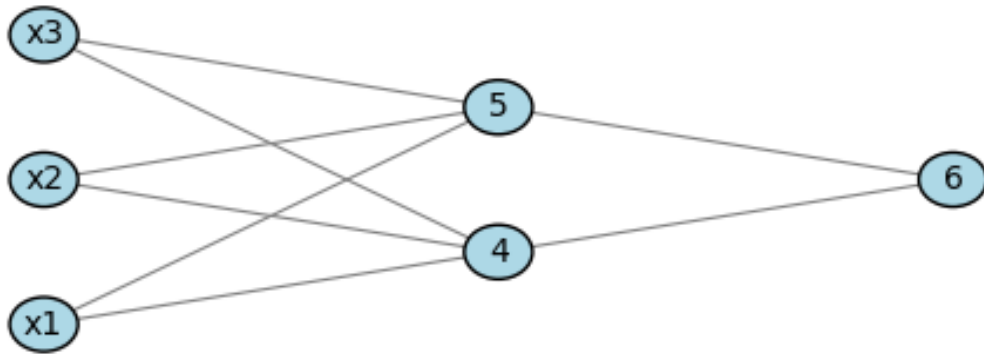
$$\Delta w_{ji} \leftarrow \eta \delta_j x_{ji} \quad (6.3)$$

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji} \quad (6.4)$$

Trong đó:

- o_k : đầu ra của node k .
- t_k : mục tiêu (target) tương ứng với node k .
- δ_k, δ_h : sai số truyền ngược ở lớp đầu ra và lớp ẩn.
- η : hệ số học (learning rate).
- x_{ji} : đầu vào đến node j từ node i .

Hình 6.5 thể hiện một mạng nơ-ron với 3 node đầu vào, 2 node ẩn và 1 node đầu ra, tương ứng với một lớp đầu vào, một lớp ẩn và một lớp đầu ra.



Hình 6.5: Một mạng nơ-ron đơn giản.

Giá trị đầu vào và trọng số ban đầu được thể hiện qua bảng 6.1.

Bảng 6.1: Giá trị đầu vào và trọng số ban đầu

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	w_{04}	w_{05}	w_{06}
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Tính toán đầu vào và đầu ra được thể hiện qua bảng 6.2.

Bảng 6.2: Tính toán đầu vào và đầu ra

Đơn vị j	Đầu vào I_j	Đầu ra O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$\frac{1}{1+e^{0.7}} = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$\frac{1}{1+e^{-0.1}} = 0.525$
6	$(-0.3)(0.332) + (-0.2)(0.525) + 0.1 = -0.105$	$\frac{1}{1+e^{0.105}} = 0.474$

Tính toán lỗi tại mỗi nút được thể hiện qua bảng 6.3.

Bảng 6.3: Tính toán lỗi tại mỗi nút

Đơn vị j	δ_j
6	$(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$
5	$(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$

Tính toán cập nhật trọng số được thể hiện qua bảng 6.4.

Bảng 6.4: Tính toán cập nhật trọng số

Trọng số	Giá trị mới
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
w_{06}	$0.1 + (0.9)(0.1311) = 0.218$
w_{05}	$0.2 + (0.9)(-0.0065) = 0.194$
w_{04}	$-0.4 + (0.9)(-0.0087) = -0.408$

6.5 Quy tắc đạo hàm của hàm Lan truyền ngược (Backpropagation)

6.5.1 Hàm lỗi

Hàm lỗi cho mẫu dữ liệu huấn luyện d được xác định như sau:

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2 \quad (6.5)$$

Trong đó, t_d là đầu ra mong muốn, và o_d là đầu ra thực tế của mạng.

6.5.2 Cập nhật trọng số với Gradient Descent

Thuật toán suy giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent) cập nhật các trọng số w_{ji} bằng cách tính đạo hàm hàm lỗi đối với mỗi trọng số và sau đó điều chỉnh chúng. Đối với mỗi ví dụ huấn luyện d , mỗi trọng số w_{ji} được cập nhật theo công thức:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad (6.6)$$

Trong đó, η là hệ số học (learning rate), và hàm lỗi tổng quát E_d được tính bằng tổng lỗi trên tất cả các đơn vị đầu ra.

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 \quad (6.7)$$

6.5.3 Đạo hàm theo trọng số

Để tiếp tục, trọng số w_{ji} có thể ảnh hưởng đến phần còn lại của mạng thông qua net_j . Do đó, chúng ta có thể sử dụng chuỗi đạo hàm (chain rule) để viết:

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial \text{net}_j} x_{ji} \quad (6.8)$$

Mục tiêu tiếp theo của chúng ta là tìm một biểu thức hợp lý cho $\frac{\partial E_d}{\partial \text{net}_j}$.

6.5.4 Quy tắc huấn luyện đối với đơn vị đầu ra

Với đơn vị đầu ra, chúng ta có thể sử dụng chuỗi đạo hàm một lần nữa. Đầu tiên, chú ý rằng net_j ảnh hưởng đến mạng chỉ thông qua o_j , do đó:

$$\frac{\partial E_d}{\partial \text{net}_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \quad (6.9)$$

Tính toán $\frac{\partial E_d}{\partial o_j}$, chúng ta có:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 \quad (6.10)$$

Do đạo hàm ở phía bên phải bằng 0 đối với tất cả các đơn vị k khác j , chúng ta có:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 = -(t_j - o_j) \quad (6.11)$$

Tiếp theo, tính đạo hàm của o_j theo net_j . Đạo hàm này là đạo hàm của hàm sigmoid, do đó:

$$\frac{\partial o_j}{\partial \text{net}_j} = o_j(1 - o_j) \quad (6.12)$$

Kết hợp các biểu thức trên vào 6.9, ta có:

$$\frac{\partial E_d}{\partial \text{net}_j} = -(t_j - o_j) o_j(1 - o_j) \quad (6.13)$$

Kết hợp công thức này với 6.6 và 6.8, ta có quy tắc cập nhật trọng số cho các đơn vị đầu ra:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j(1 - o_j) x_{ji} \quad (6.14)$$

Quy tắc huấn luyện này chính là quy tắc cập nhật trọng số được triển khai trong thuật toán lan truyền ngược.

6.5.5 Quy tắc huấn luyện đối với trọng số của đơn vị ẩn

Đối với trường hợp đơn vị ẩn, việc suy diễn quy tắc huấn luyện phải tính đến cách thức gián tiếp mà w_{ji} ảnh hưởng đến các đầu ra của mạng và do đó ảnh hưởng đến E_d .

Để giải quyết điều này, ta sẽ tham khảo tập hợp tất cả các đơn vị ngay sau đơn vị j trong mạng. Tập hợp này được ký hiệu là $\text{Downstream}(j)$.

Chú ý rằng net_j chỉ ảnh hưởng đến các đầu ra của mạng (và do đó ảnh hưởng đến E_d) thông qua các đơn vị trong $\text{Downstream}(j)$. Do đó, ta có thể viết:

$$\frac{\partial E_d}{\partial \text{net}_j} = \sum_{k \in \text{Downstream}(j)} \frac{\partial E_d}{\partial \text{net}_k} \frac{\partial \text{net}_k}{\partial \text{net}_j} = \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial \text{net}_k}{\partial \text{net}_j} \quad (6.15)$$

$$= \sum_{k \in \text{Downstream}(j)} -\delta_k \cdot \frac{\partial \text{net}_k}{\partial o_j} \cdot \frac{\partial o_j}{\partial \text{net}_j} \quad (6.16)$$

Vì $\text{net}_k = \sum_j w_{kj} o_j$, nên:

$$\frac{\partial \text{net}_k}{\partial o_j} = w_{kj}$$

Và nếu hàm kích hoạt là sigmoid: $o_j = \sigma(\text{net}_j)$ thì:

$$\frac{\partial o_j}{\partial \text{net}_j} = o_j(1 - o_j)$$

Thay vào biểu thức 6.16, ta được:

$$\frac{\partial E_d}{\partial \text{net}_j} = \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} o_j(1 - o_j) \quad (6.17)$$

Sắp xếp lại các hạng tử và sử dụng δ_j để ký hiệu cho $-\frac{\partial E_d}{\partial \text{net}_j}$, ta có:

$$\delta_j = o_j(1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj} \quad (6.18)$$

Cuối cùng, quy tắc cập nhật trọng số đối với đơn vị ẩn là:

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (6.19)$$

6.5.6 Tính toán $E[z_t]$

Ta có công thức định nghĩa z_t từ công thức 3.13:

$$z_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \xi_t \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}. \quad (6.20)$$

Ta tính kỳ vọng của z_t dựa trên tính chất của ξ_t .

1. Tính kỳ vọng của ξ_t :

$$P(\xi_t = 0) = 1 - \pi_t, \quad P(\xi_t = \pi_t^{-1}) = \pi_t.$$

$$E[\xi_t] = 0 \cdot (1 - \pi_t) + \pi_t^{-1} \cdot \pi_t = 1.$$

2. Tính kỳ vọng của z_t : Áp dụng tính tuyến tính của kỳ vọng:

$$E[z_t] = E \left[\frac{\partial f}{\partial w_h} + \xi_t \frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h} \right].$$

Vì $\frac{\partial f}{\partial w_h}$ và $\frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}$ là các đại lượng xác định tại thời điểm t , ta có:

$$E[z_t] = \frac{\partial f}{\partial w_h} + E[\xi_t] \cdot \frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

Thay $E[\xi_t] = 1$:

$$E[z_t] = \frac{\partial f}{\partial w_h} + \frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

3. Liên hệ với $\frac{\partial h_t}{\partial w_h}$: Theo quy tắc đạo hàm hàm hợp, công thức 3.5:

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}. \quad (6.21)$$

So sánh với kỳ vọng $E[z_t]$, ta thấy rằng:

$$E[z_t] = \frac{\partial h_t}{\partial w_h}.$$

Việc sử dụng ξ_t với $E[\xi_t] = 1$ đảm bảo rằng z_t là một **ước lượng không chệch** (unbiased estimator) của gradient đầy đủ $\frac{\partial h_t}{\partial w_h}$, dù quá trình lan truyền ngược có thể bị cắt ngắn ngẫu nhiên. Điều này cho phép giảm chi phí tính toán mà không làm thay đổi kỳ vọng của gradient.

6.5.7 Các đại lượng xác định (không ngẫu nhiên)

Các đại lượng $\frac{\partial f}{\partial w_h}$ và $\frac{\partial f}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}$ được coi là **xác định (deterministic)** tại thời điểm t vì những lý do sau:

1. Tính chất của hàm f và trạng thái h_{t-1} : Hàm $f(x_t, h_{t-1}, w_h)$ là hàm tường minh mô tả sự phụ thuộc của trạng thái hiện tại h_t vào:

- Đầu vào x_t (cố định tại bước t),
- Trạng thái trước đó h_{t-1} (đã được tính ở bước $t-1$),
- Tham số w_h (không đổi trong quá trình lan truyền ngược).

Do đó, các đạo hàm riêng $\frac{\partial f}{\partial w_h}$ và $\frac{\partial f}{\partial h_{t-1}}$ chỉ phụ thuộc vào các giá trị **đã biết** tại thời điểm t .

2. Tính chất của $\frac{\partial h_{t-1}}{\partial w_h}$: Biểu thức này là gradient của trạng thái h_{t-1} theo tham số w_h , được tính từ bước trước theo công thức đệ quy. Tại thời điểm t , giá trị này:

- Hoặc đã được lưu từ bước trước nếu quá trình không bị cắt ($\xi_{t-1} \neq 0$),
- Hoặc bằng 0 nếu quá trình lan truyền bị cắt tại $t - 1$ ($\xi_{t-1} = 0$).

Dù là trường hợp nào, giá trị này vẫn **xác định** tại thời điểm t .

3. Sự khác biệt với ξ_t : Biến duy nhất mang tính **ngẫu nhiên** trong công thức của z_t là ξ_t , được sinh ra tại thời điểm t để quyết định việc có tiếp tục lan truyền gradient hay không. Các đại lượng còn lại đã được xác định trước khi ξ_t được chọn.

Ví dụ minh họa (tại $t = 3$):

- Đầu vào x_3 và trạng thái h_2 đã biết.
- Tính $\frac{\partial f(x_3, h_2, w_h)}{\partial w_h}$ và $\frac{\partial f(x_3, h_2, w_h)}{\partial h_2}$: các đạo hàm này phụ thuộc vào dữ liệu cố định.
- $\frac{\partial h_2}{\partial w_h}$ đã được tính tại bước $t = 2$, có thể bằng 0 nếu $\xi_2 = 0$.
- Duy nhất ξ_3 là ngẫu nhiên và quyết định việc có sử dụng $\frac{\partial h_2}{\partial w_h}$ trong z_3 hay không.

Do đó:

- **Không ngẫu nhiên:** Các đạo hàm $\frac{\partial f}{\partial w_h}$, $\frac{\partial f}{\partial h_{t-1}}$, $\frac{\partial h_{t-1}}{\partial w_h}$ đều phụ thuộc vào thông tin đã xác định tại thời điểm t .
- **Ngẫu nhiên:** Duy nhất ξ_t là biến ngẫu nhiên điều khiển tính không chệch của ước lượng z_t .

Điều này đảm bảo rằng $E[z_t] = \frac{\partial h_t}{\partial w_h}$, tức là z_t là một **ước lượng không chệch** của gradient, giúp giảm chi phí tính toán mà vẫn đảm bảo tính đúng đắn về kỳ vọng.

(Backpropagation Through Time - BPTT)

6.5.8 Mở rộng cơ sở toán: Lan truyền ngược theo thời gian trong RNN

Phần này trình bày cách tính gradient của hàm mục tiêu đối với tất cả các tham số mô hình đã được tách riêng.

Để đơn giản, xét một mạng hồi tiếp (RNN) không có tham số độ chệch (bias), với hàm kích hoạt lớp ẩn là ánh xạ đồng nhất $\phi(x) = x$. Tại mỗi bước thời gian t , đầu vào và mục tiêu của một mẫu đơn lần lượt là:

$$x_t \in \mathbb{R}^d, \quad y_t \in \mathbb{R}^q.$$

Trạng thái ẩn $h_t \in \mathbb{R}^h$ và đầu ra $o_t \in \mathbb{R}^q$ được xác định theo công thức:

$$h_t = \mathbf{W}_{hx}x_t + \mathbf{W}_{hh}h_{t-1}, \quad (6.22)$$

$$o_t = \mathbf{W}_{qh}h_t \quad (6.23)$$

trong đó:

- $\mathbf{W}_{hx} \in \mathbb{R}^{h \times d}$ là trọng số kết nối từ đầu vào đến lớp ẩn.
- $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ là trọng số hồi tiếp của lớp ẩn.
- $\mathbf{W}_{qh} \in \mathbb{R}^{q \times h}$ là trọng số từ lớp ẩn đến đầu ra.

Giả sử $l(o_t, y_t)$ là hàm mất mát tại thời điểm t , hàm mục tiêu tổng quát qua T bước thời gian được định nghĩa là:

$$L = \frac{1}{T} \sum_{t=1}^T l(o_t, y_t) \quad (6.24)$$

Phụ thuộc của các biến trong mạng: Từ các phương trình trên, ta thấy rằng việc tính toán trạng thái ẩn h_3 tại bước thời gian thứ 3 phụ thuộc vào:

- Tham số mô hình: $\mathbf{W}_{hx}$, $\mathbf{W}_{hh}$,
- Trạng thái ẩn trước đó: h_2 ,
- Đầu vào hiện tại: x_3 .

Lan truyền ngược theo thời gian trong RNN tuyến tính

Trong quá trình huấn luyện, ta cần tính gradient đối với các tham số này:

$$\frac{\partial L}{\partial \mathbf{W}_{hx}}, \quad \frac{\partial L}{\partial \mathbf{W}_{hh}}, \quad \frac{\partial L}{\partial \mathbf{W}_{qh}}.$$

Chúng ta có thể thực hiện lan truyền ngược theo chiều ngược lại và lưu trữ các gradient. Để diễn tả linh hoạt phép nhân giữa ma trận, vector và vô hướng có hình dạng khác nhau theo quy tắc đạo hàm hàm hợp, ta sử dụng toán tử prod .

Bước 1: Gradient theo đầu ra Việc lấy đạo hàm của hàm mục tiêu đối với đầu ra mô hình tại bất kỳ bước thời gian t là:

$$\frac{\partial L}{\partial \mathbf{o}_t} = \frac{1}{T} \frac{\partial l(\mathbf{o}_t, y_t)}{\partial \mathbf{o}_t} \in \mathbb{R}^q \quad (6.25)$$

Bước 2: Gradient theo trọng số đầu ra $\mathbf{W}_{\text{dh}}$ Hàm mục tiêu phụ thuộc vào $\mathbf{W}_{\text{dh}}$ thông qua tất cả các đầu ra $\mathbf{o}_1, \dots, \mathbf{o}_T$. Áp dụng quy tắc đạo hàm hàm hợp:

$$\frac{\partial L}{\partial \mathbf{W}_{\text{dh}}} = \sum_{t=1}^T \text{prod} \left(\frac{\partial L}{\partial \mathbf{o}_t}, \frac{\partial \mathbf{o}_t}{\partial \mathbf{W}_{\text{dh}}} \right) = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{o}_t} \mathbf{h}_t^\top \quad (6.26)$$

Bước 3: Gradient theo trạng thái ẩn $\mathbf{h}_t$ Tại thời điểm cuối T , L chỉ phụ thuộc vào $\mathbf{h}_T$ qua $\mathbf{o}_T$, do đó:

$$\frac{\partial L}{\partial \mathbf{h}_T} = \text{prod} \left(\frac{\partial L}{\partial \mathbf{o}_T}, \frac{\partial \mathbf{o}_T}{\partial \mathbf{h}_T} \right) = \mathbf{W}_{\text{dh}}^\top \frac{\partial L}{\partial \mathbf{o}_T} \quad (6.27)$$

Đối với bất kỳ bước thời gian $t < T$ nào, việc tính toán trở nên phức tạp hơn do hàm mục tiêu L phụ thuộc vào $\mathbf{h}_t$ thông qua hai con đường:

1. Trạng thái ẩn tiếp theo $\mathbf{h}_{t+1}$
2. Đầu ra hiện tại $\mathbf{o}_t$

Theo **quy tắc đạo hàm hàm hợp**, gradient của trạng thái ẩn $\frac{\partial L}{\partial \mathbf{h}_t} \in \mathbb{R}^h$ tại bước $t < T$ có thể được tính **đệ quy** như sau:

$$\frac{\partial L}{\partial \mathbf{h}_t} = \text{prod} \left(\frac{\partial L}{\partial \mathbf{h}_{t+1}}, \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} \right) + \text{prod} \left(\frac{\partial L}{\partial \mathbf{o}_t}, \frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t} \right) = \mathbf{W}_{hh}^\top \frac{\partial L}{\partial \mathbf{h}_{t+1}} + \mathbf{W}_{qh}^\top \frac{\partial L}{\partial \mathbf{o}_t} \quad (6.28)$$

$$\frac{\partial L}{\partial \mathbf{h}_t} = \mathbf{W}_{hh}^\top \frac{\partial L}{\partial \mathbf{h}_{t+1}} + \mathbf{W}_{dh}^\top \frac{\partial L}{\partial \mathbf{o}_t} \quad (6.29)$$

Tập trung vào các vấn đề số học như gradient triệt tiêu/bùng nổ và cách tính gradient cho các tham số $\mathbf{W}_{hx}$ và $\mathbf{W}_{hh}$.

Để phân tích, việc mở rộng tính toán đệ quy cho bất kỳ bước thời gian $1 \leq t \leq T$ cho ta:

$$\frac{\partial L}{\partial \mathbf{h}_t} = \sum_{i=t}^T (\mathbf{W}_{hh}^\top)^{T-i} \mathbf{W}_{qh}^\top \frac{\partial L}{\partial \sigma_{T+i-t}} \quad (6.30)$$

Từ công thức 6.30, chúng ta có thể thấy rằng ngay cả ví dụ tuyến tính đơn giản này đã làm nổi bật một số vấn đề quan trọng của mô hình chuỗi dài: nó liên quan đến lũy thừa rất lớn của $\mathbf{W}_{hh}^\top$. Trong đó, các giá trị riêng nhỏ hơn 1 sẽ làm cho gradient triệt tiêu, còn các giá trị riêng lớn hơn 1 sẽ làm cho gradient tăng không kiểm soát. Điều này gây ra bất ổn định số học, thể hiện qua hiện tượng gradient triệt tiêu hoặc bùng nổ. Một cách để giải quyết là cắt ngắn chuỗi thời gian đến một kích thước phù hợp về mặt tính toán. Trong thực tế, việc cắt ngắn này cũng có thể được thực hiện bằng cách tách gradient sau một số bước thời gian nhất định. Về sau, các mô hình chuỗi phức tạp hơn như LSTM có thể giảm thiểu vấn đề này.

Hàm mục tiêu L phụ thuộc vào các tham số mô hình $\mathbf{W}_{hx}$ và $\mathbf{W}_{hh}$ trong lớp ẩn thông qua các trạng thái ẩn $\mathbf{h}_1, \dots, \mathbf{h}_T$. Để tính gradient đối với các tham số này $\partial L / \partial \mathbf{W}_{hx} \in \mathbb{R}^{h \times d}$ và $\partial L / \partial \mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$, chúng ta áp dụng quy tắc đạo hàm hàm hợp:

$$\frac{\partial L}{\partial \mathbf{W}_{hx}} = \sum_{t=1}^T \text{prod} \left(\frac{\partial L}{\partial \mathbf{h}_t}, \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hx}} \right) = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{x}_t^\top \quad (6.31)$$

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \text{prod} \left(\frac{\partial L}{\partial \mathbf{h}_t}, \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} \right) = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top \quad (6.32)$$

trong đó $\partial L / \partial \mathbf{h}_t$ được tính toán đệ quy bởi 6.27 và 6.29, là đại lượng chính ảnh hưởng đến sự ổn định số học.

Do lan truyền ngược qua thời gian là một ứng dụng của lan truyền ngược trong RNN, quá trình huấn luyện RNN luân phiên giữa lan truyền thuận và lan truyền ngược qua thời gian. Hơn nữa, lan truyền ngược qua thời gian tính toán và lưu trữ các gradient trên theo thứ tự. Cụ thể, các giá trị trung gian được lưu trữ sẽ được tái sử dụng để tránh tính toán trùng lặp, chẳng hạn như lưu $\partial L / \partial \mathbf{h}_t$ để sử dụng trong cả $\partial L / \partial \mathbf{W}_{hx}$ và $\partial L / \partial \mathbf{W}_{hh}$.

Giải thích ngắn gọn

- **Gradient triệt tiêu/bùng nổ:** Xảy ra do tích lũy các lũy thừa của $\mathbf{W}_{hh}^\top$, phụ thuộc vào giá trị riêng của nó.
- **Cách giải quyết:** Cắt ngắn chuỗi (truncation) hoặc sử dụng kiến trúc phức tạp hơn như LSTM.
- **Tính gradient cho $\mathbf{W}_{hx}$ và $\mathbf{W}_{hh}$:** Dựa trên trạng thái ẩn $\mathbf{h}_t$ và đầu vào $\mathbf{x}_t$ hoặc trạng thái ẩn trước đó $\mathbf{h}_{t-1}$.
- **Tối ưu hóa:** Tái sử dụng giá trị trung gian (ví dụ: $\partial L / \partial \mathbf{h}_t$) để giảm chi phí tính toán.

6.5.9 Các tham số trong RNN

Mạng RNN có ba ma trận trọng số (tham số) được chia sẻ qua tất cả các bước thời gian:

- U : Biến đổi đầu vào x_t thành trạng thái ẩn s_t .
- W : Biến đổi trạng thái ẩn trước đó s_{t-1} thành trạng thái hiện tại s_t .
- V : Biến đổi trạng thái ẩn s_t thành đầu ra y_t .

Các ma trận U , W , và V đều thực hiện các **biến đổi tuyến tính** đối với đầu vào tương ứng. Hình thức cơ bản nhất của biến đổi này là **lớp fully connected (FC)**. Vì vậy, U , W , và V là các **ma trận trọng số** học được trong quá trình huấn luyện.

Chúng ta có thể định nghĩa trạng thái ẩn và đầu ra của RNN như sau:

$$s_t = f(s_{t-1}W + x_tU) \quad (6.33)$$

$$y_t = s_tV \quad (6.34)$$

Trong đó, $f(\cdot)$ là một hàm kích hoạt phi tuyến, thường là **tanh** hoặc **ReLU**.

6.5.10 Stacked RNN (RNN nhiều tầng) [7]

Ta có thể **xếp chồng nhiều tầng RNN** để tạo thành một mạng **RNN nhiều lớp** (stacked RNN hoặc multi-layer RNN).

Trong cấu trúc này, **trạng thái ẩn** s_t^l của một cell RNN tại **tầng** l và **thời điểm** t sẽ nhận:

- Đầu ra y_t^{l-1} từ cell RNN tại tầng dưới $l - 1$, và
- Trạng thái ẩn trước đó s_{t-1}^l của chính cell đó tại tầng l .

Biểu diễn công thức:

$$s_t^l = f(s_{t-1}^l, y_t^{l-1}) \quad (6.35)$$

Trong đó:

- f là hàm kích hoạt (thường là **tanh**, **ReLU**, hoặc **LSTM/GRU**).
- s_t^l : trạng thái ẩn tại tầng l , thời điểm t .
- y_t^{l-1} : đầu ra từ tầng trước tại cùng thời điểm.

6.5.11 Các kiểu kiến trúc RNN

RNN có thể được tổ chức theo nhiều dạng kết nối tùy theo nhiệm vụ:

- **One-to-One**: Một đầu vào $\rightarrow$ Một đầu ra (ví dụ: phân loại ảnh).
- **One-to-Many**: Một đầu vào $\rightarrow$ Chuỗi đầu ra (ví dụ: sinh nhạc từ một từ khóa).
- **Many-to-One**: Chuỗi đầu vào $\rightarrow$ Một đầu ra (ví dụ: phân loại cảm xúc từ văn bản).
- **Many-to-Many (gián tiếp)**: Chuỗi đầu vào $\rightarrow$ Chuỗi đầu ra (dịch máy, nhưng đầu ra bắt đầu sau khi toàn bộ đầu vào được xử lý).
- **Many-to-Many (trực tiếp)**: Mỗi đầu vào tương ứng với một đầu ra (ví dụ: gán nhãn từ trong câu).

Tài liệu tham khảo

- [1] I. Sutskever, “Training recurrent neural networks,” *PhD Thesis, University of Toronto*, 2013. [Online]. Available: https://www.cs.toronto.edu/~ilya/pubs/2013_sutskever_thesis.pdf
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [3] I. Sutskever, J. Martens, G. Dahl, and G. E. Hinton, “Importance of initialization and momentum in deep learning,” in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1139–1147.
- [4] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2014.
- [5] T. Tieleman and G. Hinton, “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,” in *COURSERA: Neural Networks for Machine Learning*, 2012.
- [6] S. J. Reddi, S. Kale, S. Kumar, and J. Liao, “On the convergence of adam and beyond,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1338–1346.
- [7] K. Cho, B. van Merriënboer, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

- [8] Cloudflare, “What is a neural network?” 2025, truy cập vào ngày 04/05/2025. [Online]. Available: <https://www.cloudflare.com/learning/ai/what-is-neural-network/>
- [9] J. Dancker, “A brief introduction to recurrent neural networks,” *Towards Data Science*, 2022, truy cập vào ngày 04/05/2025. [Online]. Available: <https://towardsdatascience.com/a-brief-introduction-to-recurrent-neural-networks-638f64a61ff4>
- [10] W. contributors, “Neural network (machine learning),” 2025, truy cập vào ngày 04/05/2025. [Online]. Available: [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
- [11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by backpropagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [14] P. J. Werbos, “Backpropagation through time: What it does and how to do it,” *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.
- [15] K. Contributors, “Simplernn layer,” 2025, truy cập vào ngày 04/05/2025. [Online]. Available: https://keras.io/api/layers/recurrent_layers/simple_rnn/
- [16] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014. [Online]. Available: <https://arxiv.org/abs/1406.1078>
- [17] A. Potapczynski, L. Wu, D. Biderman, G. Pleiss, and J. P. Cunningham, “Bias-free scalable gaussian processes via randomized truncations,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 2021, pp. 8609–8619. [Online]. Available: <https://proceedings.mlr.press/v139/potapczynski21a.html>

- [18] G. Yin, C. Xu, and L. Wang, “Q-learning algorithms with random truncation bounds and applications to effective parallel computing,” *Journal of Optimization Theory and Applications*, vol. 137, pp. 435–451, 2008.
- [19] B. Bjarkason, “Randomized truncated svd levenberg–marquardt approach to geothermal natural state and history matching,” *Water Resources Research*, vol. 54, no. 8, pp. 6051–6067, 2018.
- [20] D. Kim, S. Shin, K. Song, W. Kang, and I.-C. Moon, “Soft truncation: A universal training technique of score-based diffusion model for high precision score estimation,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 2022, pp. 11 201–11 228. [Online]. Available: <https://proceedings.mlr.press/v162/kim22i.html>
- [21] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” *ICML*, vol. 28, pp. 1310–1318, 2013. [Online]. Available: http://www.icml2013.org/wp-content/uploads/2013/06/icml_2013_paper_519.pdf
- [22] J. Larson, M. Menickelly, and S. M. Wild, “Derivative-free optimization methods,” *arXiv preprint arXiv:1904.11585*, 2019. [Online]. Available: <https://arxiv.org/abs/1904.11585>
- [23] R. Williams and D. Zipser, “Experimental analysis of the real-time recurrent learning algorithm,” *Connection Science*, vol. 1, no. 1, pp. 87–111, 1989. [Online]. Available: <https://doi.org/10.1080/09540098908915747>
- [24] C. Talleg and Y. Ollivier, “Unbiased online recurrent optimization,” *arXiv preprint arXiv:1702.05043*, 2017. [Online]. Available: <https://arxiv.org/abs/1702.05043>
- [25] A. Mujika, F. Meier, and A. Steger, “Approximating real-time recurrent learning with random kronecker factors,” in *Advances in Neural Information Processing Systems (NeurIPS) 31*, 2018. [Online]. Available: <https://arxiv.org/abs/1805.10842>

- [26] Y. Ollivier, C. Tallec, and G. Charpiat, “Training recurrent networks online without backtracking,” *arXiv preprint arXiv:1507.07680*, 2015. [Online]. Available: <https://arxiv.org/abs/1507.07680>
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS) 31*, 2017, pp. 5998–6008. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [28] J. Weston, S. Chopra, and A. Bordes, “Memory networks,” *arXiv preprint arXiv:1410.3916*, 2014. [Online]. Available: <https://arxiv.org/abs/1410.3916>
- [29] K. Yu, V.-K. Vinh-Hieu, D. Bui, and N. Le, “Spiking neural networks and their applications: A review,” *Scientific Reports*, vol. 11, no. 1, p. 19037, 2021. [Online]. Available: <https://www.nature.com/articles/s41598-021-98448-0>
- [30] M. Boresta, T. Colombo, A. D. Santis, and S. Lucidi, “A mixed finite differences scheme for gradient approximation,” *Journal of Optimization Theory and Applications*, vol. 194, no. 1, pp. 1–24, 2022. [Online]. Available: <https://doi.org/10.1007/s10957-021-01994-w>
- [31] A. Tuzhilin, “Neuromorphic computing: The next frontier in ai and computing,” *University of the People Blog*, 2025. [Online]. Available: <https://www.uopeople.edu/blog/neuromorphic-computing/>
- [32] Z. Liu, J. Li, X. Zhang, M. Bansal, and L. Jiang, “Recurrent transformers for sequence modeling,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 4700–4710.
- [33] A. M. Dai, Z. Yang, R. Salakhutdinov, and Q. V. Le, “Transformer-xl: Attentive language models beyond a fixed-length context,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 2978–2988.
- [34] F. Wu, X. Wu, and X. Sun, “Hybrid rnn-transformer models for sequence-to-sequence learning,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1023–1032.

- [35] H. Liu, J. Zhou, X. Liu, and D. Xu, “Hybrid transformer and rnn models for sequence prediction,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 3713–3722.
- [36] H. Yang, F. Zhang, Y. Li, and H. Hu, “Hybrid rnn-transformer networks for time series forecasting,” in *Proceedings of the 2021 International Conference on Machine Learning (ICML)*, 2021, pp. 6235–6245.
- [37] J. Li, L. Wang, H. Zheng, and Y. Liu, “Hybrid rnn-transformer architecture for speech recognition,” in *Proceedings of the 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 1632–1636.
- [38] J. Kirkpatrick *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [39] S.-A. Rebuffi, A. Kolesnikov, and C. H. Lampert, “Learning to learn without forgetting by maximizing transfer and minimizing interference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 3641–3650.